Does On-Policy Distillation Need to Complete Every Rollout?

Zhengxin Zhang[1,*], **Maosen Tang**[1,*], **Ning Wang**[1,*], **Changhao Li**[2,*], **Jingren Zhou**[3], **Bo Dai**[2], **Sainyam Galhotra**[1], **Claire Cardie**[1]

[1]Cornell University [2]Georgia Tech [3]Alibaba Group

{zz865, mt872, nw366}@cornell.edu

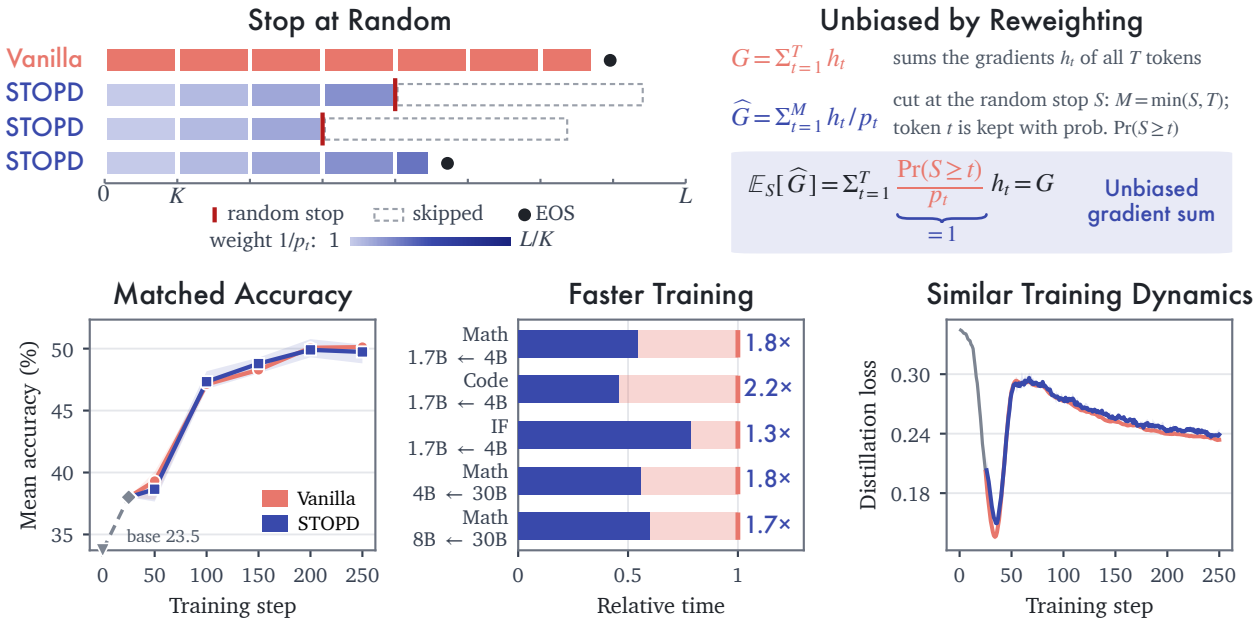


Figure 1 | Overview of STOPD: random truncation with blockwise inverse-probability weighting (top) matches Vanilla OPD's accuracy and distillation loss for an 8B student in mathematics (bottom left and right) while reducing total training time across tasks and model scales (bottom center).

Abstract | On-policy distillation (OPD) effectively transfers capabilities from a teacher to a student through feedback on student-generated rollouts. However, generating complete student rollouts and computing teacher feedback for every token make OPD computationally expensive. We introduce **S**tochastic **T**runcated **O**n-**P**olicy **D**istillation (STOPD), which accelerates OPD by randomly truncating student rollouts and correcting the resulting sampling bias with blockwise inverse-probability weighting (IPW). At each training step, we sample a stopping position for each rollout before generation and let the student generate until that position or natural termination. We obtain teacher feedback on every generated token and use blockwise IPW to compensate for the lower probability of retaining later tokens while controlling the weights. We prove that this weighting preserves Vanilla OPD's gradient sum in expectation and derive one-step update-error bounds for SGD, Adam, and Muon. We evaluate STOPD across multiple teacher–student model pairs, task domains, and optimizers. STOPD reduces generated tokens by 25–48% while maintaining accuracy comparable to Vanilla OPD, achieving 1.3–2.2× training speedups.

1. Introduction

On-policy distillation (OPD) effectively transfers capabilities from a teacher to a student through feedback on student-generated rollouts [Gu et al., 2024; Agarwal et al., 2024; Lu & Thinking Machines Lab, 2025]. This places teacher guidance on the contexts the student encounters when generating its own rollouts. However, each Vanilla OPD update generates complete student rollouts, scores every

[*]Equal contribution.

token with the teacher, and trains the student on all generated tokens [Wu et al., 2026]. As rollouts grow longer during training, both autoregressive generation and the student's forward and backward passes become more expensive. Scaling up the teacher and student further increases the cost of scoring, generation, and updates. These growing costs make generation and student updates major computational bottlenecks. These costs motivate a practical question: *does OPD need to complete every rollout to obtain an effective student update?*

Existing approaches reduce different parts of this computation. Sparse supervision selects which generated positions contribute to the loss [Xu et al., 2026; Wang et al., 2026; Sheng et al., 2026]. These methods reduce student-update time but still require complete rollout generation, which remains the dominant cost. Motivated by declining teacher guidance at later positions, Early Stopping Rollout [Zhou et al., 2026] truncates generation to a fixed prefix. Although a fixed prefix cutoff reduces generation, it omits later-token gradients, biasing the estimate of Vanilla OPD's gradient sum and changing the directions of student updates.

We introduce **S**tochastic **T**runcated **O**n-**P**olicy **D**istillation (STOPD), which accelerates OPD by randomly truncating student rollouts and correcting the resulting sampling bias with blockwise inverse-probability weighting (IPW) [McLeish, 2011]. At each training step, we sample a stopping position for each rollout before generation and let the student generate until that position or natural termination. Every generated token receives teacher feedback, and its loss is weighted by its inverse inclusion probability. However, uniformly sampled stopping positions can give late tokens extreme weights, which can destabilize training. We therefore combine IPW with blockwise truncation. We divide token positions up to the max training length into consecutive blocks and round each sampled stopping position up to the end of its block. This increases the inclusion probabilities of late tokens, thereby limiting their weights.

We show that STOPD preserves the full-generation gradient and token-count sums in expectation. We then analyze its normalized gradient, separating the variation introduced by random truncation from the remaining scale difference, and show how batch averaging reduces the stochastic discrepancy. Finally, we bound how this gradient discrepancy affects the one-step parameter update under common optimizers such as SGD, Adam, and Muon. We further analyze how STOPD saves tokens and time. STOPD saves more tokens in expectation as rollout length increases. These token savings can translate into greater training-time savings when STOPD also improves generation throughput by reducing long-context computation or freeing cache space for more concurrent rollouts.

We evaluate STOPD in multiple domains (e.g., mathematics, code, and instruction following) with students from 1.7B to 8B and teachers from 4B to 30B. STOPD generates 25–48% fewer tokens and achieves 1.3–2.2× training speedups while closely matching Vanilla OPD in accuracy. The gains persist when replacing the main AdamW optimizer with SGD or Muon. Varying the block width shows that STOPD maintains similar accuracy and computational savings over a broad range of K, while increasing the batch size from 256 to 2048 reduces truncation-induced training fluctuations without reducing the speedup. Together, these results show that OPD need not complete every rollout to learn effectively. STOPD preserves the expected gradient contributions of complete rollouts using only partial generation, achieving comparable accuracy at lower training cost. This principle could extend to other autoregressive training settings, including offline and online knowledge distillation, supervised fine-tuning, and on-policy self-distillation.

2. Related Work

On-policy distillation. OPD trains a student on its own generated rollouts using token-level teacher feedback, placing supervision on the prefixes the student encounters during generation. MiniLLM studies reverse-KL optimization [Gu et al., 2024], while Generalized Knowledge Distillation (GKD) supports flexible distribution-matching losses on student-generated sequences [Agarwal et al., 2024].

Thinking Machines Lab demonstrates OPD for mathematical reasoning and for recovering instruction-following behavior after mid-training [Lu & Thinking Machines Lab, 2025]. Subsequent variants refine the objective and the allocation of teacher feedback. DistiLLM combines skew KL with adaptive reuse of student-generated rollouts [Ko et al., 2024], and EOPD adds forward-KL supervision where teacher entropy is high [Jin et al., 2026]. FiRe filters rollouts and softly reweights their tokens [Li et al., 2026], while RG-OPD uses verifier feedback to gate distillation [Akhondzadeh et al., 2026]. OPD also extends beyond a single external teacher: MOPD integrates domain-specialized teachers [Ma et al., 2026], while OPSD uses the same model under a privileged context containing reference solutions to supervise its own rollouts [Zhao et al., 2026]. To reduce repeated computation, Lightning OPD precomputes teacher feedback on rollouts from an SFT-initialized student and reuses it during offline training [Wu et al., 2026].

Sparse supervision and rollout truncation. Sparse supervision studies which token positions should contribute to distillation. TIP uses student uncertainty and teacher–student disagreement to guide token selection [Xu et al., 2026], while TA-OPD prioritizes learnable teacher signals [Wang et al., 2026]. SelecTKD weights token losses according to whether the teacher accepts the student's proposals [Huang et al., 2025]. IER further accounts for the reliability of sampled-token gradient estimates when selecting supervision positions [Sheng et al., 2026]. Selecting loss positions alone, however, does not shorten rollout generation. ESR reduces generation by stopping at a fixed rollout prefix and distilling only that prefix [Zhou et al., 2026]. Positions beyond the cutoff have zero probability of supervision, and their missing gradient contributions are not compensated. Its prefix gradient is therefore generally biased as an estimator of the full-generation local gradient sum.

Rather than selecting loss positions on complete rollouts or imposing a fixed cutoff, we apply random truncation to student rollouts and correct unequal token coverage with inverse-probability weighting [McLeish, 2011; Beatson & Adams, 2019]. We use blockwise truncation to control extreme token weights and characterize its trade-off between generation cost and estimator variance. We preserve the expected full-generation local gradient sum while reducing generation, teacher scoring, and student-update computation.

3. Stochastic Truncated On-Policy Distillation

In this section, we present the training procedure in Section 3.1, study gradient preservation and update error in Section 3.2, and analyze token and time savings in Section 3.3.

3.1. Training Procedure

At training step k, consider a batch of B prompts $x_1, \ldots, x_B$ and the student π_{θ_k}. Let L be the max length and $1 \le T_i \le L$ the length that rollout i would have under full generation, including a naturally generated end-of-sequence token when reached. Before generating each student rollout, we randomly sample a stopping position. The student then generates until that position or natural termination. We then apply inverse-probability weighting (IPW) to the observed token losses to compensate for omitted tokens. A naive IPW implementation samples the stopping position uniformly from $\{1, \ldots, L\}$. However, long OPD rollouts can extend close to the max length, where tokens have very small probabilities of being retained. A token at position L, for example, is observed with probability $1/L$ and therefore receives weight L. When L is large, observing such a token can produce an unusually large gradient contribution.

To limit these extreme weights, we introduce blockwise truncation. We group rollout positions into consecutive blocks and round each sampled stopping position up to the end of its block. Let $K \ge 1$ denote the block width. When K divides L, the last block is retained with probability K/L, reducing the largest weight from L to L/K. Larger blocks thus trade additional generation for less extreme weighting. Before generating each rollout, we sample the anchor A_i to select the stopping block and

Algorithm 1 STOPD, with changes from Vanilla OPD in red.

Require: Student π_θ, teacher q, max length L, block width K, batch size B, warmup steps W
Run W Vanilla OPD updates and retain the student and optimizer states.
Set $C \leftarrow N_W$.
for each subsequent training step **do**
  Sample a batch of B prompts.
  for each prompt x_i **do**
    Draw A_i independently and compute S_i using Eq. (1).
    Generate until natural termination or S_i.
    Obtain local teacher feedback on every generated token.
    Assign $w_t = 1/p_t$ using Eq. (2).
  end for
  Compute $\widehat{\mathcal{L}}$ using the current denominator C.
  Apply the baseline optimizer to $\nabla_\theta \widehat{\mathcal{L}}$.
  Set $C \leftarrow \sum_i \sum_{t=1}^{M_i} w_t$ for the next step.
end for

set the cutoff to that block's end:

$$A_i \sim \text{Uniform}\{0, \ldots, L-1\}, \qquad S_i = \min\{K\,(\lfloor A_i/K \rfloor + 1)\,, L\}\,. \tag{1}$$

The student generates until natural termination or the sampled cutoff S_i, retaining $M_i = \min(T_i, S_i)$ tokens. We obtain teacher feedback on all retained tokens and weight each token's loss by the inverse probability that generation reaches its position. For $1 \le t \le T_i$, this probability is

$$p_t = \mathbb{P}(S_i \ge t) = \frac{L - K\lfloor (t-1)/K \rfloor}{L} > 0. \tag{2}$$

We can compute each token's inclusion probability and inverse-probability weight $w_t = 1/p_t$ without knowing the complete rollout length. Later positions are retained less often and receive correspondingly larger weights. Let $h_{i,t}$ denote the gradient contributed by token t in rollout i. The full gradient sum and our IPW estimate are

$$G_k = \sum_{i=1}^{B} \sum_{t=1}^{T_i} h_{i,t}, \qquad \widehat{G}_k = \sum_{i=1}^{B} \sum_{t=1}^{M_i} w_t h_{i,t}. \tag{3}$$

Vanilla computes its training loss by averaging token losses over the complete batch of $N_k = \sum_{i=1}^{B} T_i$ tokens, giving the gradient $g_k^{\mathrm{V}} = G_k / N_k$. Since random truncation leaves N_k unknown, we normalize our weighted token losses using an estimate C_k from the preceding batch:

$$\widehat{\mathcal{L}}_k(\theta) = \frac{1}{C_k} \sum_{i=1}^{B} \sum_{t=1}^{M_i} w_t \ell_{i,t}(\theta), \qquad \widehat{g}_k = \widehat{G}_k / C_k. \tag{4}$$

Here, $\ell_{i,t}(\theta)$ is the distillation loss for token t in rollout i, with $h_{i,t} = \nabla_\theta \ell_{i,t}(\theta_k)$. The weights and C_k are held constant during differentiation. Initial rollouts of base models are usually short relative to the max length, so random truncation offers limited savings. We therefore use $W \ge 1$ Vanilla OPD warmup steps to avoid truncation noise during this phase and initialize C_{W+1} with the last warmup batch's exact token count N_W. After switching to random truncation, each batch estimates its complete token count using the same inclusion weights:

$$\widehat{N}_k = \sum_{i=1}^{B} \sum_{t=1}^{M_i} w_t, \qquad C_k = \begin{cases} N_W, & k = W+1, \\ \widehat{N}_{k-1}, & k > W+1. \end{cases} \tag{5}$$

Algorithm 1 summarizes the training procedure of our method. Next, we analyze gradient preservation and update error in Section 3.2 and how our method saves tokens and time in Section 3.3.

3.2. Gradient Preservation and Update Error

We first establish gradient preservation by showing that STOPD preserves the full-generation gradient and token-count sums in expectation. We then analyze the normalized gradient, separating the variation introduced by random truncation from the remaining scale difference, and show how batch averaging reduces the stochastic discrepancy. Finally, we bound how this gradient discrepancy affects the one-step parameter update under SGD, Adam, and Muon.

We compare STOPD with full-generation Vanilla OPD at a fixed training step. We condition on the complete rollouts and their local teacher feedback, leaving only the sampled stopping positions random, and write $\mathbb{E}_S$ for expectation over this randomness. The comparison uses three conditions. (1) Given the same prompt and previously generated tokens, the next-token distribution does not depend on the remaining generation budget. (2) The local loss at a retained position depends only on the prompt, the generated tokens through that position, and its local teacher feedback, so removing the later tokens leaves the retained token's loss unchanged. (3) Conditional on the training history, stopping positions are mutually independent across rollouts and are sampled independently of the complete rollout and teacher-feedback randomness. Each retained position is weighted by its exact inverse inclusion probability $1/p_t$. Appendix A.1 states these conditions formally.

Preservation. Let $I_{i,t} = \mathbf{1}\{S_i \geq t\}$ indicate whether position t is retained. Since $\mathbb{E}_S[I_{i,t}] = p_t$, the weighted contribution of each position satisfies

$$\mathbb{E}_S\left[\frac{I_{i,t}}{p_t} h_{i,t}\right] = h_{i,t}, \qquad \mathbb{E}_S\left[\frac{I_{i,t}}{p_t}\right] = 1. \tag{6}$$

Thus inverse-probability weighting restores both the gradient contribution and the token-count contribution of every position in expectation. Equivalently, the estimators in Section 3.1 can be written as

$$\widehat{G}_k = \sum_{i=1}^{B} \sum_{t=1}^{T_i} \frac{I_{i,t}}{p_t} h_{i,t}, \qquad \widehat{N}_k = \sum_{i=1}^{B} \sum_{t=1}^{T_i} \frac{I_{i,t}}{p_t}, \tag{7}$$

and summing Eq. (6) gives

$$\mathbb{E}_S[\widehat{G}_k] = G_k, \qquad \mathbb{E}_S[\widehat{N}_k] = N_k. \tag{8}$$

Equation (8) establishes preservation at the level directly affected by truncation. The training gradient also contains the normalization in Eq. (4). Vanilla uses $g_k^{\mathrm{V}} = G_k/N_k$, whereas STOPD uses $\widehat{g}_k = \widehat{G}_k/C_k$, with C_k fixed before the current stopping positions are sampled. Therefore

$$\mathbb{E}_S[\widehat{g}_k] = \frac{G_k}{C_k} = \frac{N_k}{C_k} g_k^{\mathrm{V}}. \tag{9}$$

IPW therefore removes the bias introduced by truncating the gradient sum.

Error after normalization. Define $\rho_k = (N_k - C_k)/C_k$. Then we have

$$\underbrace{\widehat{g}_k - g_k^{\mathrm{V}}}_{\text{gradient discrepancy}} = \underbrace{\frac{\widehat{G}_k - G_k}{C_k}}_{\text{variation from random truncation}} + \underbrace{\rho_k g_k^{\mathrm{V}}}_{\text{scale difference}}. \tag{10}$$

To quantify the variation from random truncation, let $V_k = \mathbb{E}_S[\|\widehat{G}_k - G_k\|^2]$. Since Eq. (8) gives $\mathbb{E}_S[\widehat{G}_k - G_k] = 0$, taking the expected squared norm of Eq. (10) gives

$$\mathbb{E}_S\left[\|\widehat{g}_k - g_k^{\mathrm{V}}\|^2\right] = \frac{V_k}{C_k^2} + \rho_k^2 \|g_k^{\mathrm{V}}\|^2. \tag{11}$$

Appendices A.1 and A.3 derive this variation using

$$v(T) = \sum_{t=1}^{T} \left(p_t^{-1} - 1\right) \{2(T-t) + 1\}. \tag{12}$$

If $\|h_{i,t}\| \le H$ for every token, then

$$V_k \le H^2 \sum_{i=1}^{B} v(T_i), \qquad \mathbb{E}_S\left[\|\widehat{g}_k - g_k^{\mathrm{V}}\|^2\right] \le \frac{H^2 \sum_{i=1}^{B} v(T_i)}{C_k^2} + \rho_k^2 \|g_k^{\mathrm{V}}\|^2. \tag{13}$$

Batch averaging reduces the variation from random truncation. Since $C_k \ge B$ and $v(T_i) \le v(L)$,

$$\frac{V_k}{C_k^2} \le \frac{H^2 v(L)}{B}. \tag{14}$$

Large-batch agreement. We next analyze how the scale difference behaves as the batch size grows. For $k > W + 1$, Eq. (5) gives $C_k = \widehat{N}_{k-1}$. By Eq. (8), $\widehat{N}_{k-1}$ estimates the preceding complete token count. Complete rollouts are conditionally independent and identically distributed within each batch, with token gradients bounded by H. Appendices A.4 and A.5 state the formal conditions.

Let μ_j denote the conditional mean complete rollout length at step j. Then

$$\frac{N_k}{B} = \mu_k + O_{\mathbb{P}}(B^{-1/2}), \qquad \frac{C_k}{B} = \mu_{k-1} + O_{\mathbb{P}}(B^{-1/2}), \tag{15}$$

and therefore

$$\frac{N_k}{C_k} = \frac{\mu_k}{\mu_{k-1}} + O_{\mathbb{P}}(B^{-1/2}). \tag{16}$$

Combining Eq. (16) with Eq. (10) gives

$$\widehat{g}_k - g_k^{\mathrm{V}} = \left(\frac{\mu_k}{\mu_{k-1}} - 1\right) g_k^{\mathrm{V}} + O_{\mathbb{P}}(B^{-1/2}). \tag{17}$$

Thus batch averaging reduces the stochastic discrepancy at rate $B^{-1/2}$. When the adjacent conditional mean lengths agree, $\widehat{g}_k - g_k^{\mathrm{V}} = O_{\mathbb{P}}(B^{-1/2})$.

Optimizer updates. We next translate the gradient discrepancy into a bound on the one-step update error. We compare STOPD and Vanilla under the same optimizer, from the same parameters and previous optimizer state, as formalized in Appendix B.

Let $\Phi(x)$ denote the parameter increment produced from gradient input x, with the current parameters, previous optimizer state, and hyperparameters fixed. Let Q be the global norm-clipping map in Eq. (49), and write

$$\Delta\theta^{\mathrm{S}} = \Phi(Q(\widehat{g}_k)), \qquad \Delta\theta^{\mathrm{V}} = \Phi(Q(g_k^{\mathrm{V}})). \tag{18}$$

Let Λ_{opt} denote the optimizer-specific coefficient evaluated at the Vanilla reference $Q(g_k^{\mathrm{V}})$, using the largest coefficient across disjoint parameter blocks. Since Q is nonexpansive, Eq. (50) gives

$$\|\Delta\theta^{\mathrm{S}} - \Delta\theta^{\mathrm{V}}\| \le \Lambda_{\mathrm{opt}} \|\widehat{g}_k - g_k^{\mathrm{V}}\|. \tag{19}$$

Combining Eq. (19) with Eq. (11) gives

$$\mathbb{E}_S\left[\|\Delta\theta^{\mathrm{S}} - \Delta\theta^{\mathrm{V}}\|^2\right] \le \Lambda_{\mathrm{opt}}^2 \left\{\frac{V_k}{C_k^2} + \rho_k^2 \|g_k^{\mathrm{V}}\|^2\right\}. \tag{20}$$

For SGD, Eq. (51) gives $\Lambda_{\mathrm{SGD}} = \eta$. Proposition B.1 gives the Adam coefficient Λ_{A}, and Proposition B.2 gives the Muon coefficient Λ_{M}.

Together, IPW preserves the complete gradient and token-count sums in expectation, while normalization separates the gradient discrepancy into variation from random truncation and the scale difference. Batch averaging reduces the stochastic discrepancy, and changes in conditional mean rollout length between adjacent steps determine the remaining scale difference. The optimizer-specific coefficient then bounds the corresponding one-step update error.

3.3. Token and Time Savings

We first quantify token savings, then analyze how they translate into reduced generation time.

Token savings. For a complete rollout of length T, random truncation skips the token at position t with probability $1 - p_t$. Therefore, the expected number of tokens saved is $D(T) = \sum_{t=1}^{T}(1 - p_t)$. For $K \ll T$, $D(T) \approx T^2/(2L)$, so the expected fraction of tokens saved is approximately $T/(2L)$. This corresponds to about 25% expected token savings for $T = L/2$ and nearly 50% for $T = L$.

Time savings. We further show why random truncation can yield greater speedups than token savings alone would predict. During decoding, attention reads keys and values for the prompt and previously generated tokens from the key–value (KV) cache. Later tokens attend to longer cached contexts, and random truncation is more likely to skip these positions. Truncation therefore avoids forward-pass computation for skipped tokens, reduces KV-cache reads, and limits cache growth. To relate these savings to generation time, we model decoding time as a fixed cost per round plus a cost proportional to the cached context processed. We measure this cumulative context by the KV area $\mathcal{A}_i = \sum_{t=1}^{M_i}(T_i^{\mathrm{p}} + t - 1)$, where T_i^{p} is the prompt length of rollout i. Since position t is generated with probability p_t, the expected KV area is

$$\mathbb{E}_S\mathcal{A}_i = \sum_{t=1}^{T_i} p_t\,(T_i^{\mathrm{p}} + t - 1) \approx T_i^{\mathrm{p}} T_i\left(1 - \frac{T_i}{2L}\right) + \frac{T_i^2}{2} - \frac{T_i^3}{3L}, \tag{21}$$

The approximation neglects discrete and block-rounding corrections for $K \ll T_i$; Appendix C.2 derives the expression and the time model. When rollouts are much longer than their prompts, the prompt term contributes little to the KV area. A rollout reaching half the max length then saves about 25% of its tokens and 33% of its KV area in expectation. At the max length, these savings increase to about 50% and 67%, respectively. Thus, random truncation reduces the cumulative cached context by a larger fraction than the token count.

The per-rollout reductions above accumulate across the batch. Let

$$\mathcal{A} = \sum_{i=1}^{B} \mathcal{A}_i, \qquad \mathcal{A}^{\mathrm{V}} = \sum_{i=1}^{B} \mathcal{A}_i^{\mathrm{V}},$$

where $\mathcal{A}_i^{\mathrm{V}}$ is the full-generation KV area of rollout i, and let $N = \sum_i T_i$ and $N^{\mathrm{S}} = \sum_i M_i$ denote the generated-token counts of Vanilla and STOPD, respectively. The batch-level KV-area ratio decomposes as

$$\frac{\mathcal{A}^{\mathrm{V}}}{\mathbb{E}_S[\mathcal{A}]} = \underbrace{\frac{N}{\mathbb{E}_S[N^{\mathrm{S}}]}}_{\text{token-count ratio}} \qquad \underbrace{\frac{\mathcal{A}^{\mathrm{V}}/N}{\mathbb{E}_S[\mathcal{A}]/\mathbb{E}_S[N^{\mathrm{S}}]}}_{\text{mean-context-per-token ratio}}. \tag{22}$$

The first factor captures the reduction in generated tokens, while the second captures the reduction in cached context processed per generated token. When the prompt contribution is neglected, both factors are at least one.

These reductions translate into generation-time savings according to the decoding costs and scheduling. Consider the setting where prefill time is negligible compared with decoding time. When attention over long contexts dominates decoding, the model predicts that time savings track the reduction in KV area. The same relation holds when queued rollouts replace completed ones and maintain roughly constant cache occupancy at a similar level in both methods: shorter contexts allow more rollouts to generate concurrently, improving throughput. Larger training batches can help sustain this scheduling regime. In our 1.7B mathematics experiments, generating 34% fewer tokens yields a 2.0× generation speedup, exceeding the 1.5× ratio implied by token savings alone (Table 1).

If all rollouts start together, a long surviving rollout can keep the number of decoding rounds nearly unchanged. Truncation can still accelerate generation by reducing the context processed in those rounds, but the gain is smaller when fixed costs per round dominate. Random truncation also reduces teacher scoring and student loss computation, further contributing to total training-time savings. We evaluate token and time savings in Section 5.1.

4. Experimental Setup

We use the same experimental settings for Vanilla OPD and STOPD.

Models. The main setting distills Qwen3-4B-Instruct-2507 into Qwen3-1.7B-Base [Yang et al., 2025]. For scaling, we distill Qwen3-30B-A3B-Instruct-2507 into Qwen3-4B-Base and Qwen3-8B-Base in mathematics. All models run in non-thinking mode.

Data. We use Nemotron-Cascade-RL-Math [Wang et al., 2025] for mathematics, DeepCoder-Preview [Agentica, 2025] for code, and Nemotron-RL instruction following [NVIDIA, 2025] for instruction following.

Training. Our main runs use AdamW [Loshchilov & Hutter, 2019] with learning rate 10^{-6}, batch size 256, and gradient clipping at 1, for 250 steps with three seeds. Rollouts use temperature 1 and max lengths of 16,384, 8,192, and 4,096 for mathematics, code, and instruction following, respectively. STOPD uses block width $K = 128$ after 25 Vanilla warmup steps. Each run uses two GPUs: H100 for all 1.7B experiments, H200 for the 4B student, and GB200 for the 8B student.

Evaluation. We report the average score over k sampled responses per problem (avg@k). For mathematics, we report AIME24, AIME25, and AMC23 at avg@8, and MATH500 [Hendrycks et al., 2021] and Minerva [Lewkowycz et al., 2022] at avg@3, allowing up to 32,768 tokens. For code we report HumanEval+ and MBPP+ [Liu et al., 2023], two LiveCodeBench windows (2024-08 to 2025-02 and 2025-02 to 2025-05; Jain et al., 2025), and 200 held-out training-distribution problems, allowing up to 16,384 tokens. For instruction following, we report IFEval prompt-level strict accuracy [Zhou et al., 2023] at avg@4, allowing up to 8,192 tokens. All evaluations sample at temperature 0.7 and top-$p = 0.95$. Main results are averaged over three training seeds.

Timing. We measure wall-clock time per training step, including generation, teacher scoring, the student update, and other overhead.

5. Experiments

We evaluate whether STOPD reduces training time while retaining the accuracy of Vanilla OPD. Section 5.1 compares accuracy, training speed, and training dynamics over 250 steps in three domains, Section 5.2 turns to larger students, Section 5.4 varies the block width, Section 5.5 the batch size,

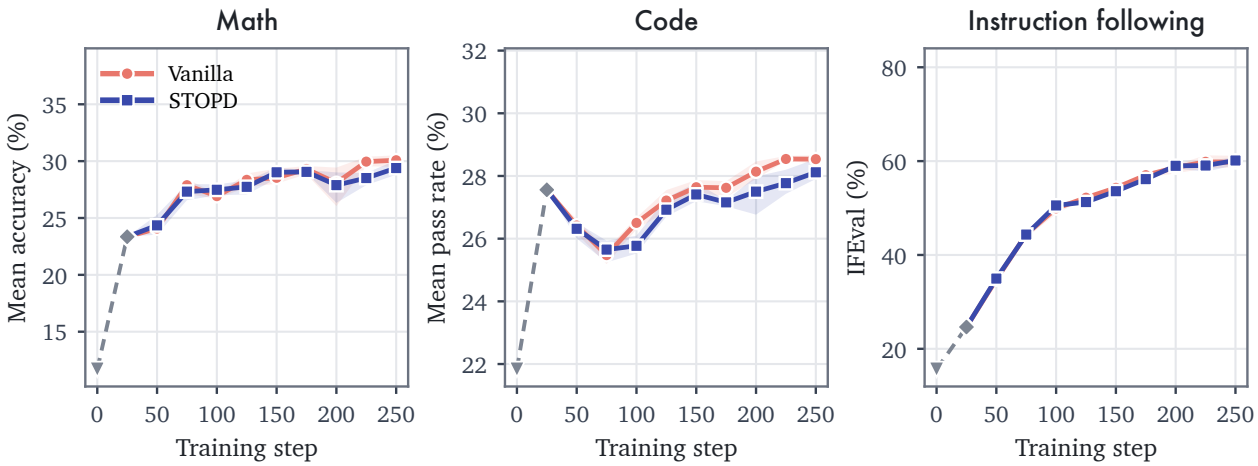


Figure 2 | Accuracy of the 1.7B student against training step, mean and range, with the base model and the shared warmup in gray.

Table 1 | Cost of a training step in the main setting.

Qwen3-1.7B-Base ← *Qwen3-4B-Instruct-2507*				Time per step (s)			
Domain	**Method**	**Tokens**	**k tok/s**	**Generation**	**Update**	**Other**	**Total**
Math	Vanilla	9,516	3.5	697	189	56	941
	STOPD	6,255	4.5	358	119	36	513
	Speedup	1.5×	1.3×	2.0×	1.6×	1.5×	**1.8**×
Code	Vanilla	6,587	4.4	383	142	41	566
	STOPD	3,444	5.6	158	78	24	260
	Speedup	1.9×	1.3×	2.4×	1.8×	1.7×	**2.2**×
Instruction following	Vanilla	710	3.8	54	36	14	104
	STOPD	533	3.8	39	29	12	80
	Speedup	1.3×	1.0×	1.4×	1.2×	1.1×	**1.3**×

and Section 5.3 repeats mathematics with SGD and Muon. Table 1 summarizes training speed for the main setting.

5.1. Main Results

Figure 2 shows that STOPD and Vanilla both increase the accuracy across all three domains. For example, student accuracy in mathematics increases from 11.7 to 30.1 with Vanilla and 29.4 with STOPD; in code, from 21.8 to 28.5 and 28.1; and in instruction following, from 15.6 to 59.8 and 60.1, respectively. Besides following similar increasing trends, STOPD and Vanilla OPD remain closely matched in accuracy across the three domains. The relative STOPD–Vanilla difference is $0.44\% \pm 1.91\%$ in mathematics, $-1.46\% \pm 0.94\%$ in code, and $0.37\% \pm 1.00\%$ in instruction following. Across all comparisons, the mean relative difference is $-0.21\% \pm 1.55\%$, and the largest absolute difference is 2.64%.

We then compare the training time of the two methods in Table 1. Overall, STOPD speeds up training by 1.8× in mathematics, 2.2× in code, and 1.3× in instruction following. STOPD generates 34%, 48%, and 25% fewer tokens in mathematics, code, and instruction following, corresponding to token-count ratios of 1.5×, 1.9×, and 1.3×. Generation is 2.0×, 2.4×, and 1.4× faster, respectively, with throughput increasing from 3.5k to 4.5k tokens/s in mathematics and from 4.4k to 5.6k tokens/s in code. The larger generation gains in mathematics and code reflect their longer rollouts: truncation reduces not only the number of generated tokens but also the cached context processed. Skipped later positions

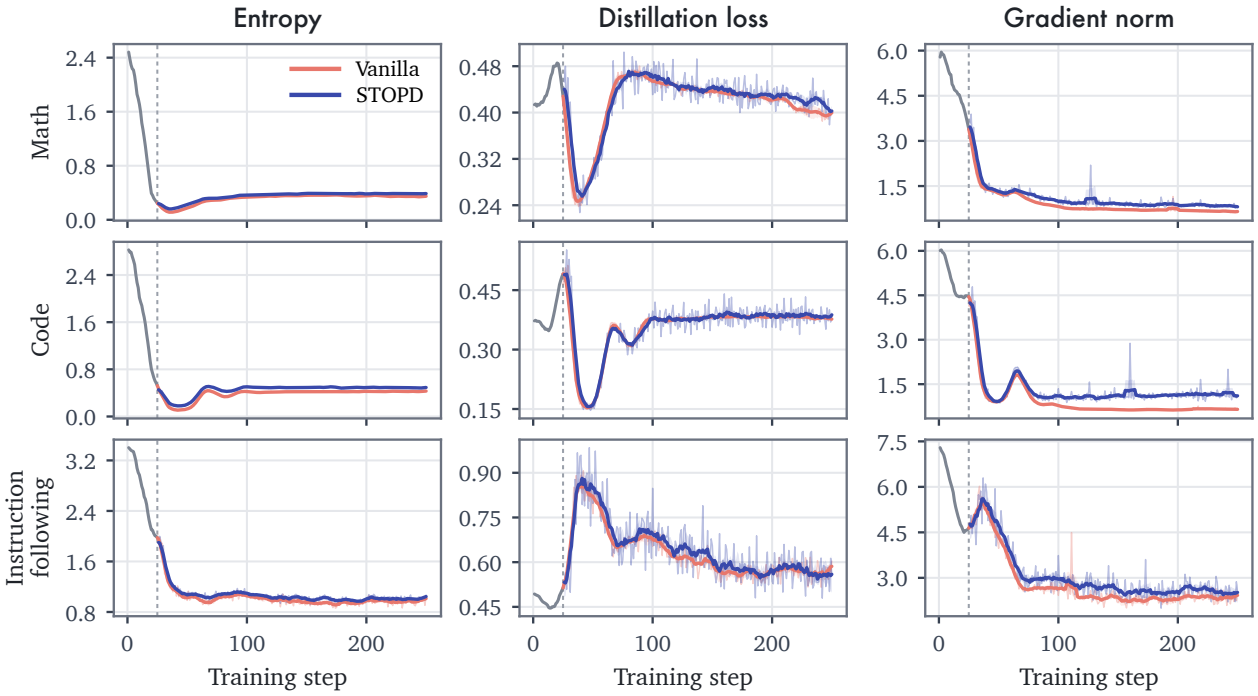


Figure 3 | Training dynamics of the 1.7B student with the shared warmup in gray.

would otherwise attend to larger KV caches, so removing them reduces KV area per generated token. In instruction following, rollouts are much shorter, so the additional KV-area reduction is smaller, leaving the generation speedup close to the token-count ratio. Student updates are also 1.2–1.8× faster, as the student's forward and backward passes process fewer tokens, and the remaining time (Other column in Table 1), mostly the forward pass that recomputes old log-probabilities, falls by 1.1–1.7× for the same reason.

Figure 3 shows that STOPD and Vanilla follow similar overall training dynamics across all three domains: the entropy, distillation-loss, and gradient-norm curves have similar shapes over training, while STOPD exhibits larger local fluctuations. These fluctuations come from the variation introduced by random truncation. Blockwise inverse-probability weighting preserves the expected contribution of truncated tokens, but positions retained with lower probability receive larger weights, so the weighted gradient varies across batches depending on which positions are retained. We provide more discussion of it in Section 5.5.

5.2. Scaling

We then evaluate STOPD at larger model scales. Figure 4 shows that both methods improve accuracy when scaling to a larger teacher and larger students, distilling Qwen3-30B-A3B-Instruct-2507 into Qwen3-4B-Base and Qwen3-8B-Base. For the 4B and 8B students, accuracy increases from 15.2% and 23.5% to 46.2% and 50.1% with Vanilla, and to 45.3% and 49.7% with STOPD, respectively. STOPD matches Vanilla more closely at larger scales. With the 30B teacher, the relative STOPD–Vanilla differences are $-0.96\% \pm 0.83\%$ for the 4B student and $-0.25\% \pm 1.01\%$ for the 8B student, with the largest absolute differences of 1.95% and 1.53%, respectively, compared with $-0.21\% \pm 1.55\%$ and 2.64% in the main 4B-teacher/1.7B-student setting (Section 5.1). Thus, scaling the teacher and student reduces both the spread and the largest observed difference from Vanilla, suggesting that STOPD is more robust and may benefit from scaling to larger models. In terms of speed, STOPD also retains its gains at larger model scales, accelerating the full training step by 1.8× for the 4B student and 1.7× for the 8B student, while generating 35% and 33% fewer tokens, respectively (Figure 4). Additional time decomposition is shown in Table 5 in Appendix E. Figure 5 further shows that the

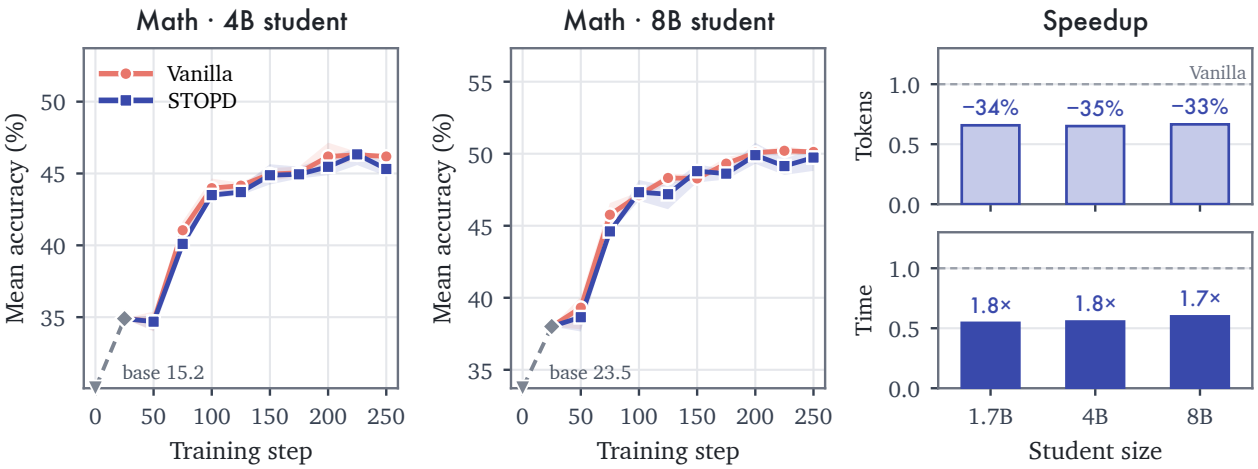


Figure 4 | Scaling in mathematics: accuracy for 4B and 8B students distilled from Qwen3-30B-A3B-Instruct-2507 (left, middle), and token and training-time speedups across student sizes, including the 1.7B main setting of Section 5.1 (right).

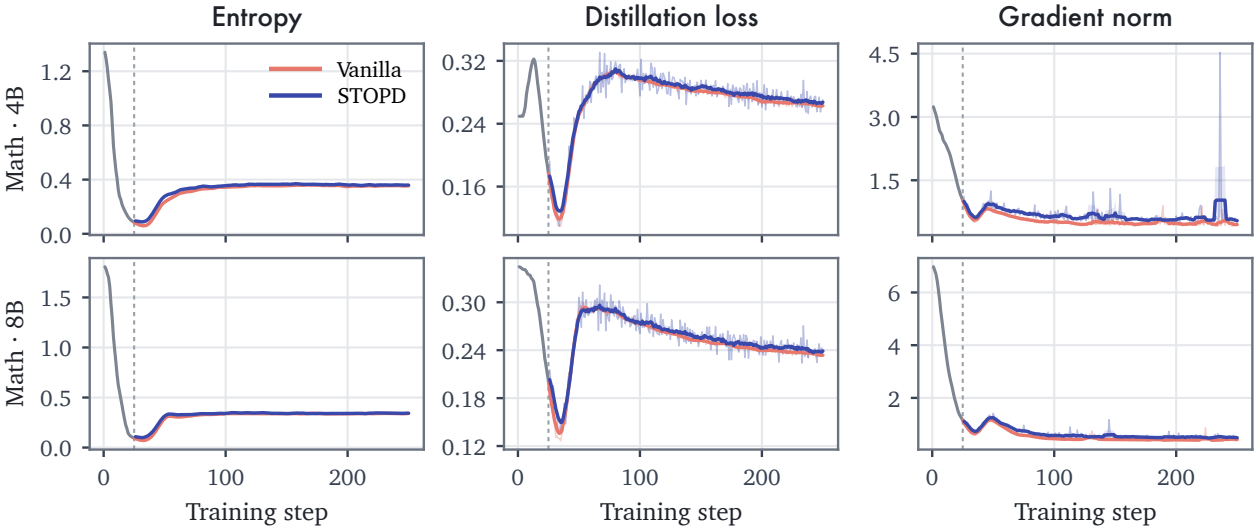


Figure 5 | Training dynamics of the 4B and 8B students in mathematics with the shared warmup in gray.

training dynamics of the larger students follow those of Vanilla.

5.3. Optimizer

Along with the main AdamW setting (Section 4), we further evaluate whether STOPD generalizes across common optimizers by testing it with SGD and Muon (Figure 6). Under SGD, accuracy increases from 11.7 to 25.6 with Vanilla and 25.5 with STOPD, with a largest absolute relative difference of 3.5%. Under Muon, it increases from 11.7 to 29.6 and 30.0, respectively, with a largest absolute relative difference of 3.0%. The computational gains also hold under both optimizers. Under SGD, STOPD generates 41% fewer tokens and accelerates the full training step by 2.1×. Under Muon, it generates 38% fewer tokens and achieves a 1.9× speedup. Additional time decomposition is shown in Table 6 in Appendix E. The training dynamics also remain similar to Vanilla under both SGD and Muon (Figure 7).

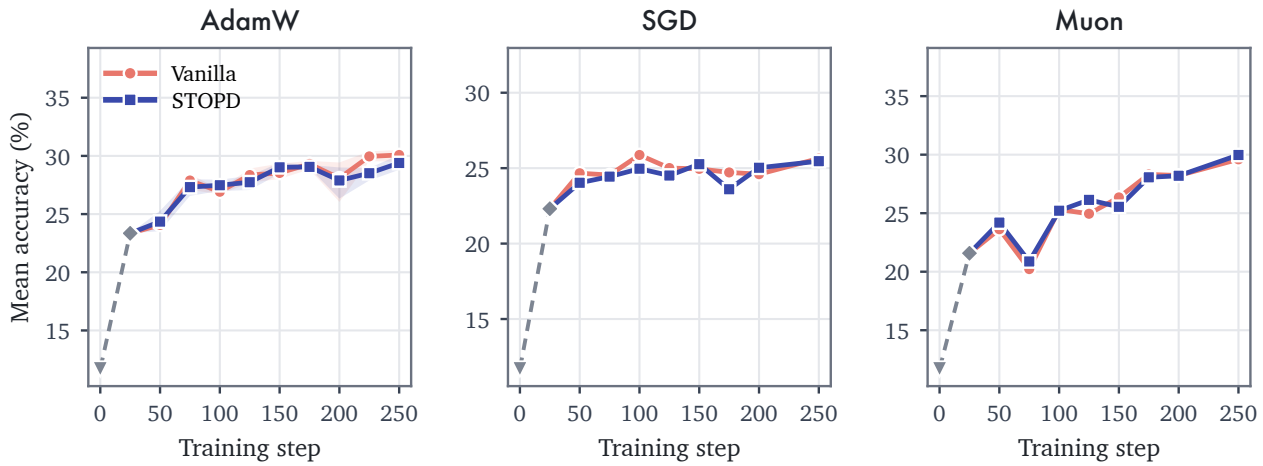


Figure 6 | Accuracy against training step in mathematics (1.7B ← 4B) under AdamW in the main setting of Section 5.1 and under SGD and Muon, with the base model and each optimizer's warmup in gray.

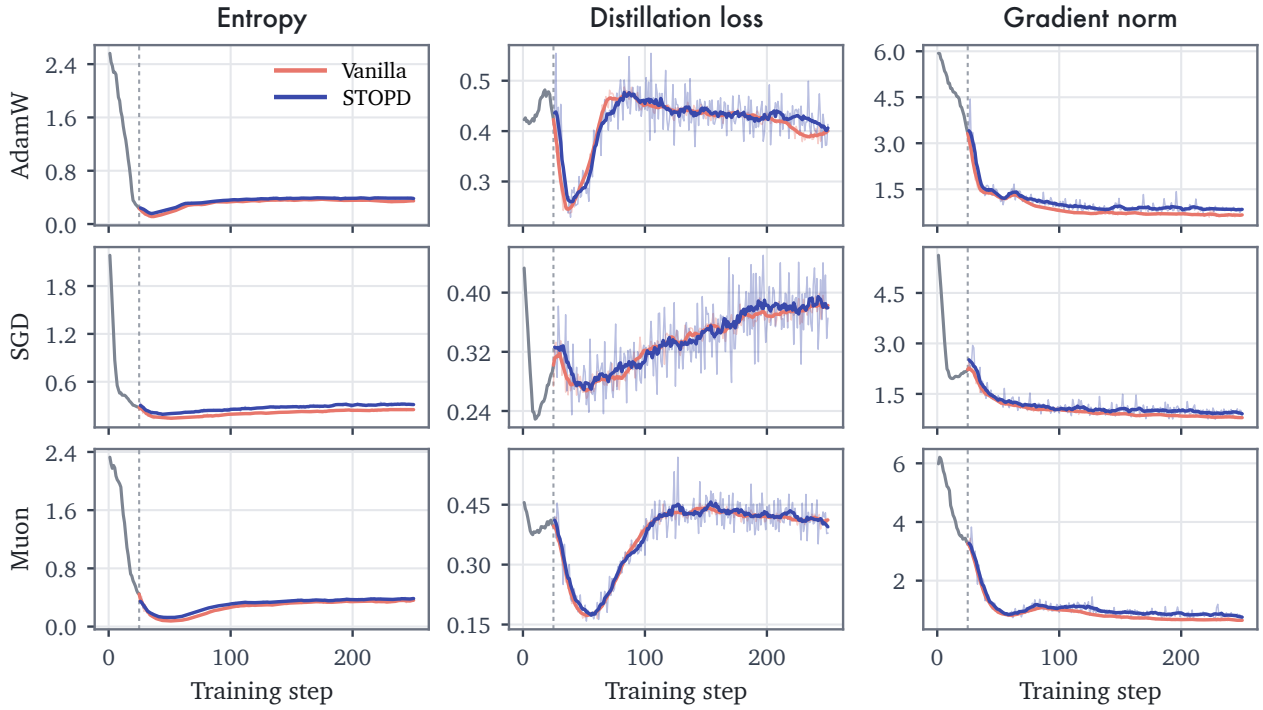


Figure 7 | Training dynamics in mathematics (1.7B ← 4B) under AdamW and under SGD and Muon, with each optimizer's warmup in gray.

5.4. Block Width

We next study how the block width K affects the trade-off between weight magnitude and computational savings. We evaluate $K \in \{32, 128, 512, 2048\}$ over three seeds in Figure 8. For $K = 32$, 128, and 512, STOPD generates 34.6%, 34.3%, and 33.1% fewer tokens, respectively, and remains 1.8× faster than Vanilla. At $K = 2048$, the saving decreases to 28.6% and the speedup to 1.7×. Accuracy is consistent across block widths. Across all pairwise comparisons among K, the mean absolute relative difference is 1.65% ± 0.97%, and the largest absolute relative difference is 4.55%. Accuracy every 50 steps and additional time decomposition are shown in Tables 3 and 7 in Appendices D and E.

We then show the largest weight per step across K and use $K = 128$ as the default. This is because it balances weight control and speedup. Reducing K to 32 raises the largest weight from 128 to 512,

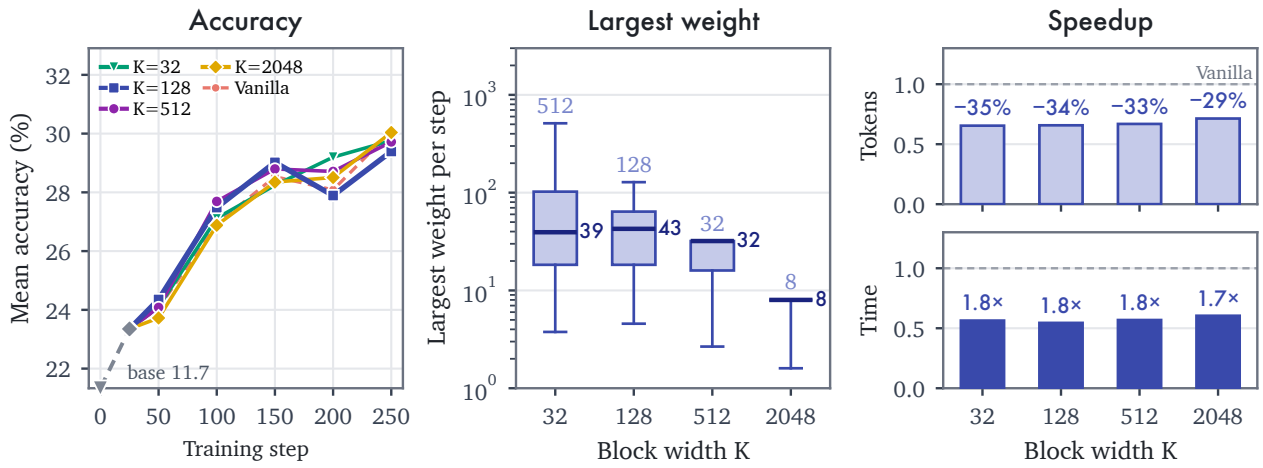


Figure 8 | Block width in mathematics (1.7B ← 4B): accuracy against training step with the base model and the shared warmup in gray (left), the largest weight in each training step as a box plot with its bound L/K above and its median beside each box (middle), and tokens and training time relative to Vanilla (right).

while the median largest weight remains similar (43 versus 39). Increasing K to 512 or 2048 lowers the weight bound further, but also reduces token savings. Thus, $K = 128$ avoids a large worst-case weight without sacrificing the savings of smaller blocks.

5.5. Batch Size

We further increase the training batch size from $B = 256$ to $B = 2048$ and train from step 5 to step 50 under the instruction-following setting with the Qwen3-4B teacher and Qwen3-1.7B student. Accuracy remains closely matched to Vanilla throughout training. From step 5 to step 50, Vanilla's accuracy increases from 16.2 to 40.5, while STOPD reaches 40.0, as shown in Table 4 in Appendix D. The relative STOPD–Vanilla difference is $-1.16\% \pm 1.07\%$, with a largest absolute difference of 2.78%. The speedup remains stable as the batch size increases from $B = 256$ to $B = 2048$, relative to Vanilla at the same batch size.

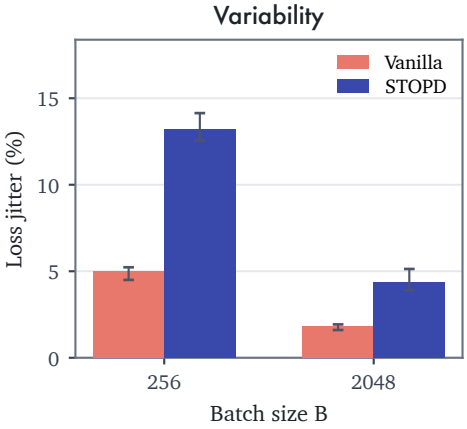


Figure 9 | Loss jitter.

Increasing the batch size also reduces the fluctuations introduced by random truncation. The loss jitter of STOPD decreases from 13.2% to 4.4%, a 67% reduction, compared with a 64% reduction for Vanilla from 5.0% to 1.8%. Because truncations are sampled independently across rollouts, larger batches average over more realizations, reducing truncation-induced gradient fluctuations at a rate of $O_{\mathbb{P}}(B^{-1/2})$. Larger batches also help sustain high generation concurrency and KV-cache occupancy, potentially enabling greater speedups.

6. Discussion

The principle behind STOPD could accelerate a broader range of autoregressive training workflows, including offline and online knowledge distillation, supervised fine-tuning, on-policy self-distillation, and multi-teacher on-policy distillation.

Knowledge Distillation (KD). In offline KD [Kim & Rush, 2016], we store the truncated sequences and their inclusion probabilities during dataset construction, then use them for subsequent student training with blockwise IPW. In online KD [Agarwal et al., 2024], we apply blockwise IPW within each

training batch using the current teacher. Both settings could reduce generation and student-update computation to speed up training.

Supervised fine-tuning. The same approach applies to SFT by randomly truncating reference responses and applying blockwise IPW to the retained next-token cross-entropy losses.

On-policy self-distillation. OPSD uses the same model for both student and teacher, with the teacher additionally conditioned on a reference solution [Zhao et al., 2026]. We can retain the complete reference solution, randomly truncate the student rollout, and apply blockwise IPW to the distillation losses computed on its retained tokens. This could also reduce computation cost.

Multi-teacher on-policy distillation. MOPD assigns each student rollout to a teacher according to its task domain [Ma et al., 2026]. We can keep this assignment, independently truncate each student rollout, and have its assigned teacher score only the retained prefix. Applying blockwise IPW preserves the expected local gradient sum within each domain and could reduce the computation needed to integrate the specialist teachers' capabilities into a shared student.

7. Conclusion

STOPD accelerates OPD by combining random rollout truncation with blockwise IPW. It preserves Vanilla OPD's gradient sum in expectation while controlling extreme token weights. We derive one-step update-error bounds for SGD, Adam, and Muon and analyze how token savings translate into training-time gains. Random truncation reduces both generated tokens and long-context computation, with additional throughput gains when long-context processing is costly or cache capacity limits concurrency. Across mathematics, code, and instruction following with 1.7B–8B students, STOPD generates 25–48% fewer tokens and achieves 1.3–2.2× training speedups while maintaining accuracy comparable to Vanilla OPD. These results show that OPD does not really need to complete every rollout to learn effectively.

References

Agarwal, R., Vieillard, N., Zhou, Y., Stanczyk, P., Ramos Garea, S., Geist, M., and Bachem, O. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, pp. 21246–21263, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/5be69a584901a26c521c2b51e40a4c20-Abstract-Conference.html.

Agentica. DeepCoder-Preview-Dataset. https://huggingface.co/datasets/agentica-org/DeepCoder-Preview-Dataset, 2025.

Akhondzadeh, M. S., Lingam, V., Tejaswi, A., Ekbote, C., Sanghavi, S., and Bojchevski, A. Reward-Gated On-Policy Distillation, 2026. URL https://arxiv.org/abs/2607.04037.

Andersson, F., Carlsson, M., and Perfekt, K.-M. Operator-Lipschitz estimates for the singular value functional calculus. *Proceedings of the American Mathematical Society*, 144(5):1867–1875, 2016. doi: 10.1090/proc/12843.

Beatson, A. and Adams, R. P. Efficient optimization of loops and limits with randomized telescoping sums. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 534–543. PMLR, 09–15 Jun 2019. URL https://proceedings.mlr.press/v97/beatson19a.html.

Boucheron, S., Lugosi, G., and Massart, P. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013. doi: 10.1093/acprof:oso/9780199535255.001.0001.

Gu, Y., Dong, L., Wei, F., and Huang, M. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=5h0qf7IBZZ.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://arxiv.org/abs/2103.03874.

Huang, H., Song, J., Zhang, Y., and Ren, P. SelecTKD: Selective Token-Weighted Knowledge Distillation for LLMs, 2025. URL https://arxiv.org/abs/2510.24021.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, pp. 58791–58831, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/94074dd5a072d28ff75a76dabed43767-Abstract-Conference.html.

Jin, W., Min, T., Yang, Y., Wei, D., Zhou, Y., Kadhe, S. R., Baracaldo, N., and Lee, K. Entropy-Aware On-Policy Distillation of Language Models, 2026. URL https://arxiv.org/abs/2603.07079.

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/. Reference implementation, revision 6456082.

Kim, Y. and Rush, A. M. Sequence-level knowledge distillation. In Su, J., Duh, K., and Carreras, X. (eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 1317–1327, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1139. URL https://aclanthology.org/D16-1139/.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

Ko, J., Kim, S., Chen, T., and Yun, S.-Y. DistiLLM: Towards Streamlined Distillation for Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24872–24895. PMLR, 2024. URL https://proceedings.mlr.press/v235/ko24c.html.

Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 3843–3857, 2022. doi: 10.52202/068431-0278. URL https://arxiv.org/abs/2206.14858.

Li, Y., Zheng, L., Yu, Y., Zhou, W., Zhong, X., Hu, X., Jin, J., Yuan, H., and Feng, T. Filter, Then Reweight: Rethinking Optimization Granularity in On-Policy Distillation, 2026. URL https://arxiv.org/abs/2606.02684.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.01210.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

Lu, K. and Thinking Machines Lab. On-Policy Distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026. URL https://thinkingmachines.ai/blog/on-policy-distillation/.

Ma, W., Wei, J., Zhao, L., Zhang, H., Xiao, B., Li, L., Yang, Q., Gao, B., Wang, Y., Li, R., Dong, J., Sui, Z., and Luo, F. MOPD: Multi-Teacher On-Policy Distillation for Capability Integration in LLM Post-Training, 2026. URL https://arxiv.org/abs/2606.30406.

McLeish, D. A general method for debiasing a Monte Carlo estimator. *Monte Carlo Methods and Applications*, 17(4), January 2011. ISSN 0929-9629. doi: 10.1515/mcma.2011.013. URL http://dx.doi.org/10.1515/mcma.2011.013.

NVIDIA. Nemotron-RL-instruction_following. https://huggingface.co/datasets/nvidia/Nemotron-RL-instruction_following, 2025.

Sheng, H., Ye, Z., Wang, H., Wang, J., Gu, J., and Kang, J. 1% of tokens can be enough: On gradient estimation in on-policy distillation, 2026. URL https://arxiv.org/abs/2609.24432.

Tallec, C. and Ollivier, Y. Unbiasing truncated backpropagation through time, 2017. URL https://arxiv.org/abs/1705.08209.

Wang, B., Lee, C., Lee, N., Lin, S.-C., Dai, W., Chen, Y., Chen, Y., Yang, Z., Liu, Z., Shoeybi, M., Catanzaro, B., and Ping, W. Nemotron-Cascade: Scaling cascaded reinforcement learning for general-purpose reasoning models. *arXiv preprint arXiv:2512.13607*, 2025. URL https://arxiv.org/abs/2512.13607.

Wang, Y., Lu, S., Gu, Y., Wang, P., Yang, Y., Yan, Z., Xie, C., Wu, J., and Yang, H. Not All Disagreement Is Learnable: Token Teachability in On-Policy Distillation, 2026. URL https://arxiv.org/abs/2605.26844.

Wu, Y., Han, S., and Cai, H. Lightning OPD: Efficient Post-Training for Large Reasoning Models with Offline On-Policy Distillation, 2026. URL https://arxiv.org/abs/2604.13010.

Xu, Y., Sang, H., Zhou, Z., He, R., Wang, Z., and Geramifard, A. TIP: Token Importance in On-Policy Distillation, 2026. URL https://arxiv.org/abs/2604.14084.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Zhao, S., Xie, Z., Liu, M., Huang, J., Pang, G., Chen, F., and Grover, A. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models, 2026. URL https://arxiv.org/abs/2601.18734.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL https://arxiv.org/abs/2311.07911.

Zhou, Z., Li, J., Tang, H., Wu, Y. N., and Terzopoulos, D. Less is more: Early stopping rollout for on-policy distillation, 2026. URL https://arxiv.org/abs/2605.27028.

A. Gradient Preservation and Normalization

We prove that random truncation preserves the expected gradient sum and bound its gradient error relative to Vanilla OPD, including the effect of lagged normalization. We also analyze position weights, the relationship between token savings and variance, and the effect of batch size. The proofs specialize survival-weighted estimation [McLeish, 2011, Section 2] and randomized telescoping [Beatson & Adams, 2019, Section 2.1 and supplementary Proposition 2.1] to finite, naturally terminating rollouts.

A.1. Conditional preservation and estimation error

Fix a training step k, batch size B, max rollout length L, and block width K. Its history $\mathcal{H}_k$ contains the current student parameters θ_k and the positive denominator C_k chosen before the current cutoffs. For each prompt, consider the hypothetical complete rollout generated until its first natural end-of-sequence token or the max length L, with length $1 \le T_i \le L$. The natural end-of-sequence token is included in this length and receives its ordinary local supervision. Let $\mathcal{Y}_k$ contain these complete rollouts and any randomness used for their teacher feedback. Throughout the fixed-step analysis, $\mathbb{E}_S$ and $\mathbb{P}_S$ denote expectation and probability conditional on $(\mathcal{H}_k, \mathcal{Y}_k)$; only the independent anchors remain random. We suppress k until the lagged-denominator analysis.

The following conditions formalize the setting in Section 3.

1. Changing the remaining generation budget does not affect the next-token distribution when the prompt and previously generated tokens are fixed.
2. We use Vanilla OPD's differentiation rule for sampled tokens and teacher targets. The differentiable local loss $\ell_{i,t}(\theta)$ depends only on the prompt, tokens up to position t, and local feedback. Random truncation leaves this loss unchanged at retained positions. Full and truncated rollouts share any randomness used for local feedback.
3. Conditional on history, the anchors are mutually independent, uniform on $\{0, \dots, L-1\}$, and independent of the complete token and feedback randomness. The method uses the exact weights $1/p_t$ and treats them and C as constants in differentiation.

We compare each observed rollout with its corresponding complete rollout. The local gradients $h_{i,t} = \nabla_\theta \ell_{i,t}(\theta_k) \in \mathbb{R}^d$ are finite. Norms below are Euclidean.

For the estimators in Eqs. (3) and (4), write

$$G = \sum_{i=1}^{B} \sum_{t=1}^{T_i} h_{i,t}, \qquad N = \sum_{i=1}^{B} T_i, \qquad g^{\mathrm{V}} = G/N, \qquad \widehat{g} = \widehat{G}/C.$$

Use the stopping rule in Eq. (1) and let $I_{i,t} = \mathbf{1}\{S_i \ge t\}$ indicate whether position t is retained. Then $\mathbb{E}_S I_{i,t} = p_t$, with p_t given in Eq. (2). To express the variance, define

$$\alpha_b = \frac{LK}{(L-bK)(L-(b-1)K)} \quad (1 \le bK < L), \qquad \upsilon(T) = \sum_{b=1}^{\lfloor (T-1)/K \rfloor} \alpha_b (T - bK)^2. \tag{23}$$

Empty sums equal zero, so $\upsilon(T) = 0$ when $T \le K$.

Theorem A.1 (Gradient preservation and error)**.** *Under the stated conditions, full and truncated generation can be coupled so that every observed local gradient equals its full-generation contribution. Consequently,*

$$\mathbb{E}_S \widehat{G} = G, \qquad \mathbb{E}_S \widehat{N} = N, \qquad \mathbb{E}_S \widehat{g} = \frac{N}{C} g^{\mathrm{V}}. \tag{24}$$

For $\mathcal{V} := \mathbb{E}_S \|\widehat{G} - G\|^2$ and every $C > 0$, the implemented estimator satisfies

$$\boxed{\mathbb{E}_S\|\widehat{g} - g^{\mathrm{V}}\|^2 = \frac{\mathcal{V}}{C^2} + \left(\frac{N-C}{C}\right)^2 \|g^{\mathrm{V}}\|^2.} \tag{25}$$

If $\|h_{i,t}\| \le H$ for every token in the batch, then

$$\mathbb{E}_S\|\widehat{g} - g^{\mathrm{V}}\|^2 \le \frac{H^2 \sum_i v(T_i)}{C^2} + \left(\frac{N-C}{C}\right)^2 \|g^{\mathrm{V}}\|^2. \tag{26}$$

The variance bound $\mathcal{V} \le H^2 \sum_i v(T_i)$ is attained when, within each rollout, all local gradients equal a fixed vector of norm H.

Proof. Couple full and truncated generation from the same prompt using the same randomness for token sampling and local feedback. While the truncated run continues, identical tokens generated so far imply the same next-token distribution by budget independence, and the shared draw produces the same next token. By induction, the two runs generate identical tokens through position M_i, including a natural end-of-sequence token when reached. At each retained position, removing later tokens leaves the differentiable local loss function unchanged, so Vanilla OPD's differentiation rule gives the same gradient $h_{i,t}$ in both runs. Conditional on $(\mathcal{H}_k, \mathcal{Y}_k)$, these gradients are fixed; only their inclusion depends on the random cutoffs, as in the conditioning argument of Tallec & Ollivier [2017, Proposition 1 and Section 8]. For $t \le T_i$, Eq. (2) gives $\mathbb{E}_S I_{i,t} = p_t$, including when the final block is shorter than K. Therefore

$$\widehat{G} = \sum_{i=1}^{B} \sum_{t=1}^{T_i} \frac{I_{i,t}}{p_t} h_{i,t}, \qquad \mathbb{E}_S \widehat{G} = \sum_{i=1}^{B} \sum_{t=1}^{T_i} \frac{\mathbb{E}_S I_{i,t}}{p_t} h_{i,t} = G.$$

Taking scalar contributions equal to 1 gives $\mathbb{E}_S \widehat{N} = N$, and dividing by the fixed C gives $\mathbb{E}_S \widehat{g} = G/C = (N/C) g^{\mathrm{V}}$, proving Eq. (24). Natural termination fixes which positions exist, while their inclusion probabilities depend only on t, L, K; the weights therefore do not require the complete lengths.

To quantify STOPD's gradient error relative to Vanilla, center $\widehat{g}$ at its conditional mean G/C:

$$\widehat{g} - g^{\mathrm{V}} = \frac{\widehat{G} - G}{C} + \left(\frac{G}{C} - \frac{G}{N}\right) = \frac{\widehat{G} - G}{C} + \frac{N-C}{C} g^{\mathrm{V}}. \tag{27}$$

Here N, C, g^{V} are fixed under $\mathbb{E}_S$ and $\mathbb{E}_S(\widehat{G} - G) = 0$. Expanding the squared norm therefore gives

$$\begin{aligned} \mathbb{E}_S\|\widehat{g} - g^{\mathrm{V}}\|^2 &= \frac{\mathbb{E}_S\|\widehat{G} - G\|^2}{C^2} + \left(\frac{N-C}{C}\right)^2 \|g^{\mathrm{V}}\|^2 \\ &\quad + \frac{2(N-C)}{C^2} \left\langle \mathbb{E}_S(\widehat{G} - G), g^{\mathrm{V}} \right\rangle \\ &= \frac{\mathcal{V}}{C^2} + \left(\frac{N-C}{C}\right)^2 \|g^{\mathrm{V}}\|^2, \end{aligned}$$

proving Eq. (25). It remains to evaluate and bound $\mathcal{V}$.

Write the centered error of rollout i as

$$e_i = \sum_{t=1}^{T_i} \left(\frac{I_{i,t}}{p_t} - 1\right) h_{i,t}, \qquad \widehat{G} - G = \sum_{i=1}^{B} e_i, \qquad \mathbb{E}_S e_i = 0.$$

Each e_i depends only on its own anchor under the fixed conditioning. Thus, for $i \ne j$, $\mathbb{E}_S\langle e_i, e_j\rangle = \langle \mathbb{E}_S e_i, \mathbb{E}_S e_j\rangle = 0$ by anchor independence. This reduces the batch variance to $\mathcal{V} = \sum_i \mathbb{E}_S\|e_i\|^2$.

The indicators within a rollout share a cutoff: $I_{i,s}I_{i,t} = \mathbf{1}\{S_i \geq \max(s,t)\}$. Hence

$$\mathbb{E}_S\left[\left(\frac{I_{i,s}}{p_s}-1\right)\left(\frac{I_{i,t}}{p_t}-1\right)\right] = \frac{p_{\max(s,t)}}{p_s p_t} - 1 = \frac{1}{p_{\min(s,t)}} - 1. \tag{28}$$

To express $\mathbb{E}_S\|e_i\|^2$ in terms of gradient tails, group token pairs by the block boundaries they both cross. The reciprocal survival probability changes at $bK+1$ by

$$\frac{1}{p_{bK+1}} - \frac{1}{p_{bK}} = \frac{L}{L-bK} - \frac{L}{L-(b-1)K} = \alpha_b, \qquad 1 \leq bK < L.$$

Since $p_t^{-1} = 1$ on the first block and is constant within each block, telescoping gives

$$\frac{1}{p_{\min(s,t)}} - 1 = \sum_{\substack{b\geq 1 \\ bK<\min(s,t)}} \alpha_b.$$

The coefficient α_b contributes exactly to pairs with both positions beyond bK. Expanding $\mathbb{E}_S\|e_i\|^2$ and regrouping these pairs yields

$$\begin{aligned}
\mathbb{E}_S\|e_i\|^2 &= \sum_{s,t=1}^{T_i}\left(\frac{1}{p_{\min(s,t)}}-1\right)\langle h_{i,s}, h_{i,t}\rangle \\
&= \sum_{b=1}^{\lfloor (T_i-1)/K\rfloor} \alpha_b \sum_{s,t=bK+1}^{T_i} \langle h_{i,s}, h_{i,t}\rangle \\
&= \sum_{b=1}^{\lfloor (T_i-1)/K\rfloor} \alpha_b \left\| \sum_{t=bK+1}^{T_i} h_{i,t}\right\|^2 .
\end{aligned} \tag{29}$$

Summing over rollouts gives the exact batch variance. For the weighted count, each token contributes the scalar 1, so its tail sum is $T_i - bK$ and the same identity gives

$$\begin{aligned}
\mathcal{V} &= \sum_{i=1}^{B}\mathbb{E}_S\|e_i\|^2 = \sum_{i=1}^{B}\sum_{b=1}^{\lfloor (T_i-1)/K\rfloor}\alpha_b\left\|\sum_{t=bK+1}^{T_i} h_{i,t}\right\|^2, \\
\mathrm{Var}_S(\widehat{N}) &= \sum_{i=1}^{B}\sum_{b=1}^{\lfloor (T_i-1)/K\rfloor}\alpha_b(T_i-bK)^2 = \sum_{i=1}^{B} v(T_i).
\end{aligned} \tag{30}$$

Under $\|h_{i,t}\| \leq H$, each tail satisfies $\|\sum_{t=bK+1}^{T_i} h_{i,t}\| \leq \sum_{t=bK+1}^{T_i}\|h_{i,t}\| \leq H(T_i - bK)$. Since $\alpha_b > 0$, Eq. (30) gives

$$\mathcal{V} \leq H^2 \sum_{i=1}^{B}\sum_{b=1}^{\lfloor (T_i-1)/K\rfloor}\alpha_b(T_i-bK)^2 = H^2\sum_{i=1}^{B} v(T_i). \tag{31}$$

Substitution into the normalized-error decomposition yields

$$\mathbb{E}_S\|\widehat{g} - g^{\mathrm{V}}\|^2 \leq \frac{H^2\sum_i v(T_i)}{C^2} + \left(\frac{N-C}{C}\right)^2\|g^{\mathrm{V}}\|^2,$$

which is Eq. (26). When all gradients within each rollout equal a fixed vector of norm H, every tail has norm $H(T_i - bK)$, attaining the variance bound. □

A.2. Position weights and token coverage

Inverse-probability weighting compensates for how often each position is retained. A token at position t is retained with probability p_t, so assigning it a deterministic weight c_t gives an expected gradient contribution of $p_t c_t h_{i,t}$. Matching Vanilla OPD for arbitrary token gradients requires $p_t c_t = 1$. This condition is necessary by considering a single nonzero token gradient and sufficient by linearity. Thus $c_t = 1/p_t$ is the only deterministic position weight that preserves every token's expected contribution. Since $p_t > 0$ in Eq. (2), all these weights are finite.

We next calculate how often a position receives supervision in a batch. Among $n \geq 1$ fixed complete rollouts that reach position t, independent random truncation retains that position with probability p_t in each rollout. The number retained is therefore $Q_t \sim \text{Binomial}(n, p_t)$, giving

$$\mathbb{P}_S(Q_t \geq 1) = 1 - (1 - p_t)^n. \tag{32}$$

To retain position t at least once with probability at least $1 - \delta$, solve $(1 - p_t)^n \leq \delta$. For $0 < p_t < 1$ and $\delta \in (0, 1)$, this requires

$$\mathbb{P}_S(Q_t \geq 1) \geq 1 - \delta \quad \Longleftrightarrow \quad n \geq \left\lceil \frac{\log(1/\delta)}{-\log(1 - p_t)} \right\rceil. \tag{33}$$

Here n counts rollouts that contain the position under full generation. Positions in the first block have $p_t = 1$ and are retained in all such rollouts.

A.3. Truncation variance and block width

Random truncation introduces variation into the weighted gradient sum. For a fixed rollout of length T, $v(T)$ from Eq. (23) is the variance of its weighted token-count estimate, and $D(T)$ is the expected token saving derived in Appendix C.1. For every $1 \leq T \leq L$, these quantities satisfy

$$v(T) \leq 2TD(T), \qquad \mathcal{V} \leq 2H^2 \sum_i T_i D(T_i) \quad \text{if } \|h_{i,t}\| \leq H. \tag{34}$$

The bound connects expected token savings to gradient-sum variance through the rollout lengths.

To derive the bound, note that all tokens in a rollout share one stopping position, so their inclusion events are correlated. Applying the covariance formula in Eq. (28) to the weighted token count and grouping pairs by their smaller position gives

$$v(T) = \sum_{t=1}^{T} \frac{1 - p_t}{p_t} \{2(T - t) + 1\}.$$

Here, $2(T - t) + 1$ counts the ordered pairs whose smaller position is t. Since $K\lfloor (t-1)/K \rfloor \leq t - 1$ and $T \leq L$, the inclusion probability satisfies

$$p_t \geq \frac{L - t + 1}{L} \geq \frac{T - t + 1}{T}, \qquad \frac{2(T - t) + 1}{p_t} \leq T \frac{2(T - t) + 1}{T - t + 1} \leq 2T.$$

Multiplying by $1 - p_t$ and summing proves $v(T) \leq 2T \sum_t (1 - p_t) = 2TD(T)$. Substituting this result into the gradient-variance bound in Eq. (31) gives the second inequality in Eq. (34).

For rollouts reaching the max length, the effect of block width can be computed explicitly. Let $L = mK$, so there are m equal blocks, and write $\mathsf{H}_m = \sum_{j=1}^{m} 1/j$. The variance of the weighted count relative to L^2 is

$$\frac{v(L)}{L^2} = 1 - \frac{\mathsf{H}_m}{m}. \tag{35}$$

Setting $L = mK$ in Eq. (23) gives $\alpha_b(L - bK)^2/L^2 = (m - b)/[m(m - b + 1)]$, hence

$$\frac{v(L)}{L^2} = \frac{1}{m}\sum_{b=1}^{m-1}\frac{m-b}{m-b+1} = \frac{1}{m}\sum_{j=2}^{m}\left(1-\frac{1}{j}\right) = 1 - \frac{\mathrm{H}_m}{m}.$$

For these max-length rollouts, larger blocks reduce the variance of the weighted count at the cost of more generation (Appendix C.1). With one block, this variance is zero; as the number of blocks grows, $v(L)/L^2$ approaches 1. The corresponding relative standard deviation is $\sqrt{1 - \mathrm{H}_m/m}$, which is 0.99, 0.98, 0.93, and 0.81 for the block widths $K = 32$, 128, 512, and 2048 of Section 5.4.

A.4. Error under lagged normalization

The normalization contribution in Theorem A.1 is $\rho_k g_k^{\mathrm{V}}$, where $\rho_k = (N_k - C_k)/C_k$. For STOPD's lagged denominator, $C_{W+1} = N_W$ after $W \geq 1$ Vanilla OPD updates, and $C_k = \widehat{N}_{k-1}$ for $k > W + 1$. We bound ρ_k by comparing N_k/B and C_k/B with the conditional mean rollout lengths of their respective batches. This separates fluctuations due to rollout sampling and cutoffs from changes in the mean rollout length between updates.

For $j \in \{k-1, k\}$, let $\mathcal{H}_j$ be the history before sampling batch j, including its current student parameters. Assume additionally that, conditional on $\mathcal{H}_j$, the complete rollouts within batch j are independent and identically distributed. The anchors have the independence specified in Theorem A.1. The current count N_k sums complete lengths $T_{k,i} \in [1, L]$. To express the lagged count as a sum of rollout-level quantities, define the weighted rollout count and its upper bound by

$$\widehat{T}_{j,i} = \sum_{t=1}^{T_{j,i}} \frac{I_{j,i,t}}{p_t}, \qquad U_{L,K} = \sum_{t=1}^{L}\frac{1}{p_t}. \tag{36}$$

For a truncated batch, $\widehat{N}_j/B = B^{-1}\sum_i \widehat{T}_{j,i}$ is an average of independent and identically distributed counts conditional on $\mathcal{H}_j$. The first token is retained with weight one, giving $\widehat{T}_{j,i} \geq 1$; the bounds $I_{j,i,t} \leq 1$ and $T_{j,i} \leq L$ give $\widehat{T}_{j,i} \leq U_{L,K}$. Summing the lower bounds gives $C_k \geq B$, also for the complete lengths used at initialization.

The concentration of these averages depends on the means and variances of their rollout-level counts. For complete lengths, write

$$\mu_j = \mathbb{E}[T_{j,i} \mid \mathcal{H}_j], \qquad \sigma_j^2 = \mathrm{Var}(T_{j,i} \mid \mathcal{H}_j). \tag{37}$$

Conditional on $(\mathcal{H}_j, T_{j,i})$, the weighted count depends only on its independent anchor. Equations (24) and (30), applied to one rollout, therefore give conditional mean $T_{j,i}$ and variance $v(T_{j,i})$. Averaging over the rollout length, the laws of total expectation and total variance give

$$\begin{aligned}\mathbb{E}[\widehat{T}_{j,i} \mid \mathcal{H}_j] &= \mathbb{E}[T_{j,i} \mid \mathcal{H}_j] = \mu_j,\\ \mathrm{Var}(\widehat{T}_{j,i} \mid \mathcal{H}_j) &= \sigma_j^2 + \mathbb{E}[v(T_{j,i}) \mid \mathcal{H}_j] =: \tau_j^2.\end{aligned} \tag{38}$$

Thus weighting preserves the conditional mean count and adds the expected cutoff variance to the rollout-length variance. The corresponding conditional centers are μ_k for N_k/B and μ_{k-1} for C_k/B. These centers may differ under adaptively chosen policies; write $\Delta_{\mu,k} = |\mu_k - \mu_{k-1}|$ for this mean-length drift.

The centered counts $N_k - B\mu_k$ and $C_k - B\mu_{k-1}$ are sums of independent centered contributions conditional on their respective histories. The following form of Bernstein's inequality controls their deviations using the variance and range width of each summand.

Lemma A.2 (Bernstein concentration)**.** *Let $F_1, \dots, F_B$ be independent real random variables, each supported on an interval of length at most $u \geq 0$, and let $s^2 = B^{-1} \sum_i \mathrm{Var}(F_i)$. For every $x > 0$,*

$$\mathbb{P}\left(\left|\frac{1}{B}\sum_i (F_i - \mathbb{E}F_i)\right| > \sqrt{\frac{2s^2x}{B}} + \frac{ux}{3B}\right) \leq 2e^{-x}. \tag{39}$$

The statement also holds conditional on a history under which its assumptions hold.

Proof. Write $F_i^\circ = F_i - \mathbb{E}F_i$, so $\sum_i \mathbb{E}(F_i^\circ)^2 = Bs^2$. If $s^2 = 0$, every centered summand is zero almost surely and the result follows. Otherwise $u > 0$. The moment form of Bernstein's inequality [Boucheron et al., 2013, Theorem 2.10] gives the one-sided threshold $\sqrt{2Bs^2x} + cx$ with failure probability e^{-x} whenever, for some scale $c > 0$,

$$\sum_{i=1}^{B} \mathbb{E}|F_i^\circ|^m \leq \frac{m!}{2} Bs^2 c^{m-2}, \qquad m = 3, 4, \dots.$$

This absolute-moment condition suffices for both signs, since it bounds the positive-part moments in the cited theorem. To verify it here, each mean lies in its supporting interval, so $|F_i^\circ| \leq u$. Consequently,

$$\sum_{i=1}^{B} \mathbb{E}|F_i^\circ|^m \leq u^{m-2} \sum_{i=1}^{B} \mathbb{E}(F_i^\circ)^2 = u^{m-2} Bs^2 \leq \frac{m!}{2} Bs^2 \left(\frac{u}{3}\right)^{m-2},$$

where $m! \geq 2 \cdot 3^{m-2}$ gives the last inequality. Thus the scale $c = u/3$ satisfies the required condition. Applying the one-sided bound to both signs and taking a union bound gives

$$\mathbb{P}\left(\left|\sum_i F_i^\circ\right| > \sqrt{2Bs^2x} + \frac{ux}{3}\right) \leq 2e^{-x}.$$

Dividing the sum and its threshold by B yields the average-deviation bound in Eq. (39), since

$$\frac{\sqrt{2Bs^2x} + ux/3}{B} = \sqrt{\frac{2s^2x}{B}} + \frac{ux}{3B}.$$

Conditional on a history satisfying the lemma's assumptions, the same argument applies with its conditional means and variances. □

To apply Lemma A.2 to the two counts, denote its deviation threshold by

$$\mathfrak{r}_B(s^2, u, x) := \sqrt{\frac{2s^2x}{B}} + \frac{ux}{3B}, \qquad s^2, u \geq 0, \quad x > 0. \tag{40}$$

Choosing $x = \log(4/\delta)$ gives $2e^{-x} = \delta/2$ for any $\delta \in (0, 1)$, assigning failure probability at most $\delta/2$ to each count bound. The next theorem substitutes the two counts' respective variances and range widths into this threshold and combines their deviations with the mean-length drift.

Theorem A.3 (Finite-batch error with lagged normalization)**.** *Under these additional sampling assumptions, fix a step $k \geq W+1$ and $\delta \in (0,1)$, and set $x = \log(4/\delta)$. Define the deviation radii for N_k/B and C_k/B by*

$$\begin{aligned} r_{N,k} &= \mathfrak{r}_B(\sigma_k^2, L-1, x), && k \geq W+1, \\ r_{C,k} &= \mathfrak{r}_B(\tau_{k-1}^2, U_{L,K}-1, x), && k > W+1. \end{aligned} \tag{41}$$

At the first truncated step, $C_{W+1} = N_W$, so use $r_{C,W+1} = \mathfrak{r}_B(\sigma_W^2, L-1, x)$. With probability at least $1-\delta$, both

$$\left|\frac{N_k}{B} - \mu_k\right| \leq r_{N,k}, \qquad \left|\frac{C_k}{B} - \mu_{k-1}\right| \leq r_{C,k} \tag{42}$$

hold. These deviations and the mean-length drift bound the relative normalization error on the same event:

$$|\rho_k| = \frac{|N_k - C_k|}{C_k} \leq \frac{\Delta_{\mu,k} + r_{N,k} + r_{C,k}}{C_k/B}. \tag{43}$$

Combining this normalization control with the current cutoff error gives a bound on the gradient discrepancy. If $\|h_{k,i,t}\| \leq H$ almost surely for the current batch, then for any $\zeta > 0$ satisfying $\delta + \zeta < 1$, the following holds with probability at least $1-\delta-\zeta$:

$$\|\widehat{g}_k - g_k^{\mathrm{V}}\| \leq \frac{\dfrac{H}{B}\sqrt{\dfrac{\sum_i v(T_{k,i})}{\zeta}} + (\Delta_{\mu,k} + r_{N,k} + r_{C,k})\|g_k^{\mathrm{V}}\|}{C_k/B}. \tag{44}$$

Proof of Theorem A.3. We first apply Lemma A.2 to the two batch averages using their rollout-level moments. For $N_k = \sum_i T_{k,i}$, condition on $\mathcal{H}_k$: its summands have mean μ_k, variance σ_k^2, and range $[1, L]$. For $k > W+1$, the denominator is $C_k = \sum_i \widehat{T}_{k-1,i}$; conditional on $\mathcal{H}_{k-1}$, Eq. (38) gives summand mean μ_{k-1} and variance τ_{k-1}^2, with range $[1, U_{L,K}]$. At the first truncated step, $C_{W+1} = N_W$, whose summands are complete lengths with variance σ_W^2 and range $[1, L]$. These choices give $r_{N,k}$ and the two cases of $r_{C,k}$. Taking $x = \log(4/\delta)$ assigns failure probability $\delta/2$ to each count bound:

$$\begin{aligned} \mathbb{P}\left(\left|\frac{N_k}{B} - \mu_k\right| > r_{N,k} \,\middle|\, \mathcal{H}_k\right) &\leq \frac{\delta}{2}, \\ \mathbb{P}\left(\left|\frac{C_k}{B} - \mu_{k-1}\right| > r_{C,k} \,\middle|\, \mathcal{H}_{k-1}\right) &\leq \frac{\delta}{2}. \end{aligned}$$

Averaging over each respective history gives an unconditional failure probability at most $\delta/2$ for each event. A union bound then proves Eq. (42).

On this joint count event, the difference between the two averages is bounded by their deviations from their own means and the gap between those means:

$$\begin{aligned} \frac{|N_k - C_k|}{B} &\leq \left|\frac{N_k}{B} - \mu_k\right| + |\mu_k - \mu_{k-1}| + \left|\mu_{k-1} - \frac{C_k}{B}\right| \\ &\leq r_{N,k} + \Delta_{\mu,k} + r_{C,k}. \end{aligned}$$

Dividing by $C_k/B \geq 1$ gives Eq. (43), which controls the normalization contribution to the gradient error.

For the remaining cutoff contribution, condition on $(\mathcal{H}_k, \mathcal{Y}_k)$ so that the current complete rollouts and their local gradients are fixed. Equation (31) bounds the conditional second moment of $\widehat{G}_k - G_k$ by $H^2 \sum_i v(T_{k,i})$. When this bound is positive, Markov's inequality at the squared-norm threshold $H^2 \sum_i v(T_{k,i})/\zeta$ gives

$$\mathbb{P}\left(\|\widehat{G}_k - G_k\|^2 > \frac{H^2 \sum_i v(T_{k,i})}{\zeta} \,\middle|\, \mathcal{H}_k, \mathcal{Y}_k \right) \le \frac{\mathbb{E}[\|\widehat{G}_k - G_k\|^2 \mid \mathcal{H}_k, \mathcal{Y}_k]}{H^2 \sum_i v(T_{k,i})/\zeta} \le \zeta.$$

Taking square roots on the complementary event yields, with conditional probability at least $1 - \zeta$,

$$\|\widehat{G}_k - G_k\| \le H \sqrt{\frac{\sum_i v(T_{k,i})}{\zeta}}.$$

This also holds when the second-moment bound is zero, since the error is then zero almost surely.

Averaging this conditional failure probability and intersecting with the count event gives probability at least $1 - \delta - \zeta$. On this intersection, the normalized-error identity in Eq. (27) combines the two contributions:

$$\begin{aligned} \|\widehat{g}_k - g_k^{\mathrm{V}}\| &\le \frac{\|\widehat{G}_k - G_k\| + |N_k - C_k| \, \|g_k^{\mathrm{V}}\|}{C_k} \\ &\le \frac{H\sqrt{\sum_i v(T_{k,i})/\zeta} + B(\Delta_{\mu,k} + r_{N,k} + r_{C,k})\|g_k^{\mathrm{V}}\|}{C_k}. \end{aligned}$$

Dividing the numerator and denominator by B proves Eq. (44). □

A.5. Batch size and gradient error

Larger batches reduce the variation from random truncation and the relative sampling error in the token-count estimate. Since C_k comes from the preceding batch, changes in mean rollout length can still introduce a scale difference. The following corollary separates these effects and shows when our gradient approaches Vanilla OPD's gradient.

Corollary A.4 (Large-batch local agreement)**.** *Keep L, K, H fixed and increase the batch size B in both the current batch and its preceding normalization batch. Assume the sampling conditions of Theorem A.3 and $\|h_{k,i,t}\| \le H$ almost surely for the current gradients. The histories and conditional means may depend on B. Then*

$$\widehat{g}_k - g_k^{\mathrm{V}} = \left(\frac{\mu_k}{\mu_{k-1}} - 1 \right) g_k^{\mathrm{V}} + O_{\mathbb{P}}(B^{-1/2}). \tag{45}$$

Here the stochastic order concerns the Euclidean norm over the joint training and sampling randomness, with constants depending only on the fixed bounds. If $\mu_k/\mu_{k-1} \to 1$ in probability, the local discrepancy converges to zero in probability. When the adjacent conditional mean lengths agree, its order is $B^{-1/2}$.

Proof. We first bound the variation in the current token count and the preceding batch's token-count estimate. For fixed L, K, each rollout's complete and weighted counts are bounded independently of B, so their conditional variances are uniformly bounded over training histories. At any fixed failure probability, Theorem A.3 then gives

$$N_k/B = \mu_k + O_{\mathbb{P}}(B^{-1/2}), \qquad C_k/B = \mu_{k-1} + O_{\mathbb{P}}(B^{-1/2}).$$

Thus N_k/B estimates the current mean length μ_k, while C_k/B estimates the preceding mean length μ_{k-1}. To compare the normalization ratio with the ratio of these means, write

$$\frac{N_k}{C_k} - \frac{\mu_k}{\mu_{k-1}} = \frac{N_k/B - \mu_k}{C_k/B} + \frac{\mu_k(\mu_{k-1} - C_k/B)}{(C_k/B)\mu_{k-1}}.$$

Since $C_k/B \geq 1$ and $1 \leq \mu_j \leq L$, the two terms on the right are $O_{\mathbb{P}}(B^{-1/2})$.

We next bound the variation from random truncation. Equation (31) gives

$$\mathbb{E}_{S_k}\left\|\frac{\widehat{G}_k - G_k}{C_k}\right\|^2 \leq \frac{H^2 \sum_i v(T_{k,i})}{C_k^2} \leq \frac{H^2 v(L)}{B},$$

where $C_k \geq B$ and $v(T_{k,i}) \leq v(L)$; the latter follows because each tail-length term in Eq. (23) is nondecreasing in the complete length. Averaging this conditional bound and applying Markov's inequality gives, for every $a > 0$,

$$\mathbb{P}\left(\left\|\frac{\widehat{G}_k - G_k}{C_k}\right\| > \frac{a}{\sqrt{B}}\right) \leq \frac{H^2 v(L)}{a^2}.$$

The truncation term is therefore $O_{\mathbb{P}}(B^{-1/2})$. Combining it with the normalization error using Eq. (27) gives

$$\widehat{g}_k - g_k^{\mathrm{V}} - \left(\frac{\mu_k}{\mu_{k-1}} - 1\right) g_k^{\mathrm{V}} = \frac{\widehat{G}_k - G_k}{C_k} + \left(\frac{N_k}{C_k} - \frac{\mu_k}{\mu_{k-1}}\right) g_k^{\mathrm{V}}.$$

Both terms on the right have order $B^{-1/2}$ in probability, using $\|g_k^{\mathrm{V}}\| \leq H$. This proves Eq. (45) and its stated consequences. □

We can also bound the normalization error for a finite batch. The estimate C_k first approximates the preceding count N_{k-1}, which can differ from the current count N_k. To bound the first difference, fix $k > W + 1$ and $\delta \in (0, 1)$, and condition on $(\mathcal{H}_{k-1}, \mathcal{Y}_{k-1})$. The preceding rollout counts $\widehat{T}_{k-1,i}$ are independent over the sampled stopping positions, with means $T_{k-1,i}$, variances $v(T_{k-1,i})$, and range width at most $U_{L,K} - 1$. Applying Lemma A.2 with average variance $B^{-1} \sum_i v(T_{k-1,i})$ and $x = \log(2/\delta)$, then multiplying by B, gives

$$|C_k - N_{k-1}| \leq q_{k-1,\delta}, \qquad q_{k-1,\delta} := \sqrt{2 \log(2/\delta) \sum_i v(T_{k-1,i})} + \frac{U_{L,K} - 1}{3} \log \frac{2}{\delta} \tag{46}$$

with conditional probability at least $1 - \delta$. On this event,

$$|C_k - N_k| \leq q_{k-1,\delta} + |N_k - N_{k-1}|, \tag{47}$$

by the triangle inequality. The first term bounds the error in estimating the preceding count; the second is the difference between the two complete counts. Since the probability bound holds for each fixed training history, it also holds after averaging over histories. At the first step with random truncation, $C_{W+1} = N_W$, so the preceding count is known exactly and its estimation error is zero.

At a fixed student policy, we can also average over the rollouts and teacher feedback. Let T be the complete rollout length and h_t its token gradients, with $\|h_t\| \leq H$. Here, $\mathbb{E}$ averages over the rollout, teacher feedback, and independent stopping position. The unbiased gradient-sum and token-count estimates in Eq. (24) give

$$g_\star := \frac{\mathbb{E}[\sum_{t=1}^T h_t]}{\mathbb{E}T} = \frac{\mathbb{E}[\sum_{t=1}^T I_t h_t / p_t]}{\mathbb{E}[\sum_{t=1}^T I_t / p_t]}. \tag{48}$$

Both methods therefore have the same population token-mean gradient, defined as the expected gradient sum divided by the expected token count.

B. Optimizer Update Error

At a fixed training step, we compare the parameter updates of STOPD and Vanilla under the same optimizer. Their gradients are $\widehat{g} = \widehat{G}/C$ and $g^{\mathrm{V}} = G/N$, respectively. Both updates start from the same parameters and previous optimizer state. We use the gradient-error decomposition from Theorem A.1 to bound the update error of STOPD relative to Vanilla for SGD, Adam, and Muon.

Condition on the training history $\mathcal{H}_k$ and hypothetical complete batch $\mathcal{Y}_k$, so only the current cutoffs remain random. For a fixed step size $\eta > 0$, let $\Phi(x)$ be the parameter increment produced from gradient input x in real arithmetic. A common decoupled weight-decay increment cancels because the two updates start from the same parameters. With $\rho = (N - C)/C$, Theorem A.1 gives

$$\mathbb{E}_S\|\widehat{g} - g^{\mathrm{V}}\|^2 = \frac{\mathcal{V}}{C^2} + \rho^2\|g^{\mathrm{V}}\|^2.$$

The two terms measure cutoff noise and the mismatch between the lagged denominator and the current complete token count.

Global norm clipping fits the same comparison. For a threshold $a_{\mathrm{clip}} > 0$ and denominator stabilizer $\delta_{\mathrm{clip}} \geq 0$, define the map on the full gradient vector by

$$Q(x) = \frac{x}{\max\{1, (\|x\| + \delta_{\mathrm{clip}})/a_{\mathrm{clip}}\}}. \tag{49}$$

This map is nonexpansive.

For an optimizer below, evaluate its coefficient at the clipped Vanilla reference $Q(g^{\mathrm{V}})$ and denote the result by Λ_{opt}. For disjoint parameter blocks, use the largest block coefficient; the full squared norm is the sum of the blockwise squared norms. The resulting update satisfies

$$\begin{aligned}\|\Phi(Q(\widehat{g})) - \Phi(Q(g^{\mathrm{V}}))\| &\leq \Lambda_{\mathrm{opt}}\|Q(\widehat{g}) - Q(g^{\mathrm{V}})\| \\ &\leq \Lambda_{\mathrm{opt}}\|\widehat{g} - g^{\mathrm{V}}\|.\end{aligned} \tag{50}$$

The reference and coefficients remain fixed under the conditioning on $(\mathcal{H}_k, \mathcal{Y}_k)$. Squaring Eq. (50), taking conditional expectations, and applying Eq. (25) bounds the mean-square update error by $\Lambda_{\mathrm{opt}}^2\{\mathcal{V}/C^2 + \rho^2\|g^{\mathrm{V}}\|^2\}$.

To verify nonexpansiveness, consider the Jacobian of Q. It is the identity in the interior of the unscaled region. In the interior of the scaled region, with $s = \|x\| > 0$, its tangential and radial eigenvalues are $a_{\mathrm{clip}}/(s + \delta_{\mathrm{clip}})$ and $a_{\mathrm{clip}}\delta_{\mathrm{clip}}/(s + \delta_{\mathrm{clip}})^2$, respectively. Both lie in $[0, 1]$ because $s + \delta_{\mathrm{clip}} \geq a_{\mathrm{clip}}$. The map is continuous at zero and at the branch boundary. Integrating its derivative along the smooth pieces of a line segment therefore gives $\|Q(x) - Q(y)\| \leq \|x - y\|$.

B.1. Stochastic gradient descent

For SGD, $\Phi(x) = -\eta x$, so the update error of STOPD relative to Vanilla is the gradient error multiplied by $\Lambda_{\mathrm{SGD}} = \eta$:

$$\|\Phi(\widehat{g}) - \Phi(g^{\mathrm{V}})\| = \eta\|\widehat{g} - g^{\mathrm{V}}\|. \tag{51}$$

The same equality holds for any input x in place of $\widehat{g}$. Taking the expected squared difference over the current cutoffs gives

$$\mathbb{E}_S\|\Phi(\widehat{g}) - \Phi(g^{\mathrm{V}})\|^2 = \eta^2\left\{\frac{\mathcal{V}}{C^2} + \rho^2\|g^{\mathrm{V}}\|^2\right\}. \tag{52}$$

Our SGD runs (Section 5.3) use neither momentum nor clipping, so their coefficient is $\Lambda_{\mathrm{SGD}} = \eta = 5 \times 10^{-4}$ at every step.

The momentum update is also affine in the current gradient. With fixed previous momentum $\boldsymbol{m}$, its update is $\boldsymbol{m}^+(x) = \beta\boldsymbol{m} + (1-\beta)x$ and $\Phi(x) = -\eta\boldsymbol{m}^+(x)$. The previous-momentum term cancels between the two inputs, leaving $\Phi(x) - \Phi(g^{\mathrm{V}}) = -\eta(1-\beta)(x - g^{\mathrm{V}})$. Its coefficient is therefore $\eta|1-\beta|$, and the squared-error equality above has multiplier $\eta^2(1-\beta)^2$. For any affine update, $\mathbb{E}_S\Phi(\widehat{g}) = \Phi(G/C)$ because $\mathbb{E}_S\widehat{g} = G/C$. The Adam and Muon maps below are nonlinear in the current input; their bounds account for the full input dependence.

B.2. Adam

Adam incorporates the current gradient into both its first moment and the second moment in its denominator. We account for this coupled dependence when bounding the update error of STOPD relative to Vanilla.

At step $k \geq 1$, let $\boldsymbol{m}$ and $\boldsymbol{q} \geq 0$ be Adam's previous first and second moments. Fix $\eta > 0$, $\beta_1, \beta_2 \in [0, 1)$, and a stabilization parameter $\epsilon_{\mathrm{A}} > 0$. For a gradient input $x \in \mathbb{R}^d$, Adam's parameter increment [Kingma & Ba, 2015, Algorithm 1] is

$$\Phi_{\mathrm{A}}(x) = -\frac{\eta}{1-\beta_1^k} \frac{\beta_1\boldsymbol{m} + (1-\beta_1)x}{\sqrt{\{\beta_2\boldsymbol{q} + (1-\beta_2)x^{\odot 2}\}/(1-\beta_2^k)} + \epsilon_{\mathrm{A}}}, \tag{53}$$

where the square, square root, and division are coordinatewise. The factors $1-\beta_1^k$ and $1-\beta_2^k$ provide Adam's moment bias correction. Each output coordinate depends on only one input coordinate, but that input changes both the numerator and denominator.

We evaluate this dependence through the scalar function

$$F(z) = \frac{a + bz}{\sqrt{c + \nu z^2} + \epsilon}, \qquad \mathfrak{L}(a, b, c, \nu, \epsilon; y) = \sup_{z \neq y} \left| \frac{F(z) - F(y)}{z - y} \right|, \tag{54}$$

with $a, y \in \mathbb{R}$, $b, \nu, \epsilon > 0$, and $c \geq 0$. The reference y is fixed. The coefficient $\mathfrak{L}$ is the largest absolute change in F per unit change from y to any other input z. For Adam's coordinate j, use

$$a = \beta_1 m_j, \quad b = 1-\beta_1, \quad c = c_j := \frac{\beta_2 q_j}{1-\beta_2^k}, \quad \nu = \frac{1-\beta_2}{1-\beta_2^k}, \quad \epsilon = \epsilon_{\mathrm{A}}, \quad y = g_j^{\mathrm{V}}.$$

Here a and c contain the previous moments; b and ν weight the current gradient and its square. These values satisfy the scalar domain conditions and give

$$\Phi_{\mathrm{A}}(x)_j = -\frac{\eta}{1-\beta_1^k} F(x_j).$$

The scalar coefficient can be computed exactly from a closed formula or a finite set of polynomial roots, as established in the proof.

Proposition B.1 (Update error with Adam)**.** *For the current batch, let $\widehat{g}$ and g^{V} denote the gradients of STOPD and Vanilla, respectively. The Adam updates in Eq. (53), from the same parameter vector and previous moments $\boldsymbol{m}, \boldsymbol{q}$, satisfy*

$$\boxed{\|\Phi_{\mathrm{A}}(\widehat{g}) - \Phi_{\mathrm{A}}(g^{\mathrm{V}})\| \leq \Lambda_{\mathrm{A}} \|\widehat{g} - g^{\mathrm{V}}\|}. \tag{55}$$

Here the coordinate coefficients and their maximum are

$$\lambda_{\mathrm{A},j} = \frac{\eta}{1-\beta_1^k} \mathfrak{L}(\beta_1 m_j, 1-\beta_1, c_j, \nu, \epsilon_{\mathrm{A}}; g_j^{\mathrm{V}}), \qquad \Lambda_{\mathrm{A}} = \max_{1 \leq j \leq d} \lambda_{\mathrm{A},j}. \tag{56}$$

These coefficients are finite and fixed conditional on the training history $\mathcal{H}_k$ and complete batch $\mathcal{Y}_k$. The same pointwise bound holds with $\widehat{g}$ replaced by any gradient input $x \in \mathbb{R}^d$.

To state the conditional mean-square bound, let $\mathcal{V}_j = \mathrm{Var}_S(\widehat{G}_j)$ *and* $\rho = (N - C)/C$. *The coordinate form of Eq.* (25) *is*

$$\mathcal{E}_j := \mathbb{E}_S(\widehat{g}_j - g_j^{\mathrm{V}})^2 = \frac{\mathcal{V}_j}{C^2} + \rho^2 (g_j^{\mathrm{V}})^2. \tag{57}$$

The variance $\mathcal{V}_j$ *is obtained from Eq.* (30) *by replacing each token gradient with its* j*th component, and* $\sum_j \mathcal{V}_j = \mathcal{V}$. *Averaging over the current cutoffs then gives*

$$\mathbb{E}_S \|\Phi_{\mathrm{A}}(\widehat{g}) - \Phi_{\mathrm{A}}(g^{\mathrm{V}})\|^2 \le \sum_j \lambda_{\mathrm{A},j}^2 \mathcal{E}_j \le \Lambda_{\mathrm{A}}^2 \left\{ \frac{\mathcal{V}}{C^2} + \rho^2 \|g^{\mathrm{V}}\|^2 \right\}. \tag{58}$$

For the globally clipped update in Eq. (50), evaluate the coefficient in Eq. (56) using $y = Q(g^{\mathrm{V}})_j$ in each coordinate.

Proof. We first compute the largest absolute secant from the fixed reference y of $F(z) = (a + bz)/(\sqrt{c + \nu z^2} + \epsilon)$, namely $\mathfrak{L}(a, b, c, \nu, \epsilon; y)$. Only z varies in this calculation; the scalar parameters and reference are fixed. The value of c determines the form of the denominator: for $c > 0$ the square root stays positive, whereas for $c = 0$ it becomes $\sqrt{\nu}\,|z|$. We compute the secant maximum in these two cases before inserting the Adam state and combining coordinates.

Suppose $c > 0$. To maximize the secant over the whole real line, introduce a coordinate that both bounds the search interval and makes the square root rational:

$$t = \frac{\sqrt{\nu}\, z}{\sqrt{c} + \sqrt{c + \nu z^2}}, \qquad t_y = \frac{\sqrt{\nu}\, y}{\sqrt{c} + \sqrt{c + \nu y^2}}.$$

The first transformation is a bijection from $\mathbb{R}$ to $(-1, 1)$; t_y is the image of the fixed reference. Its inverse and the resulting square root are

$$z(t) = \frac{2\sqrt{c}\, t}{\sqrt{\nu}(1 - t^2)}, \qquad \sqrt{c + \nu z(t)^2} = \frac{\sqrt{c}(1 + t^2)}{1 - t^2}.$$

Substitution gives rational expressions for both quantities needed in the secant:

$$F(z(t)) = \frac{a(1 - t^2) + 2b\sqrt{c}\, t/\sqrt{\nu}}{\sqrt{c}(1 + t^2) + \epsilon(1 - t^2)},$$
$$z(t) - y = \frac{2\sqrt{c}(t - t_y)(1 + t t_y)}{\sqrt{\nu}(1 - t^2)(1 - t_y^2)}.$$

Subtract $F(y) = F(z(t_y))$ in the first expression and divide by the second. Cancelling the common factor $t - t_y$ gives the exact secant

$$\frac{F(z) - F(y)}{z - y} = \frac{\mathsf{n}(t)}{\mathsf{d}(t)}, \tag{59}$$

where the numerator and denominator polynomials are

$$\mathsf{n}(t) = (1 - t^2)(1 - t_y^2) \left\{ b\left[\sqrt{c}(1 - t t_y) + \epsilon(1 + t t_y)\right] - a\sqrt{\nu}(t + t_y) \right\},$$
$$\mathsf{d}(t) = (1 + t t_y)\left[\sqrt{c}(1 + t^2) + \epsilon(1 - t^2)\right]\left[\sqrt{c}(1 + t_y^2) + \epsilon(1 - t_y^2)\right].$$

The denominator has no zero on $[-1, 1]$: $1 + t t_y \ge 1 - |t_y| > 0$, and each bracket in d is positive. Consequently n/d is continuous on the closed interval. At the omitted reference point $t = t_y$, its value is the limiting secant $F'(y)$. At $t = \pm 1$, which correspond to $z \to \pm\infty$, its value is zero. Including the

reference and endpoint values therefore leaves the supremum unchanged and makes it a maximum on $[-1, 1]$.

The limits of F at $\pm\infty$ are $\pm b/\sqrt{\nu}$, so the secant quotient is not identically zero. Its positive absolute maximum therefore occurs inside $(-1, 1)$, where the quotient is nonzero and its derivative vanishes. The stationary equation is $\mathsf{n}'\mathsf{d} - \mathsf{n}\mathsf{d}' = 0$. Since both polynomials have degree at most three, their degree-five terms cancel in this equation, leaving degree at most four. The stationary polynomial is nonzero because the quotient is nonconstant. Its roots in $(-1, 1)$ therefore form a finite nonempty set that contains a maximizer:

$$\begin{aligned} \mathcal{Z}_c &= \{t \in (-1, 1) : \mathsf{n}'(t)\mathsf{d}(t) - \mathsf{n}(t)\mathsf{d}'(t) = 0\}, \\ \mathfrak{L}(a, b, c, \nu, \epsilon; y) &= \max_{t \in \mathcal{Z}_c} \frac{|\mathsf{n}(t)|}{\mathsf{d}(t)}. \end{aligned} \tag{60}$$

To evaluate this coefficient, solve the displayed stationary polynomial, retain its real roots in $(-1, 1)$, and compare $|\mathsf{n}|/\mathsf{d}$ at those roots. A root equal to t_y is retained: cancellation has already removed the apparent division by zero in the original secant.

It remains to compute the scalar coefficient when $c = 0$. The absolute value in $F(z) = (a+bz)/(\sqrt{\nu}\,|z|+\epsilon)$ makes it useful to consider the two half-lines separately. If the reference is $y = 0$, then

$$\frac{F(z) - F(0)}{z} = \frac{b\epsilon - a\sqrt{\nu}\,\operatorname{sgn}(z)}{\epsilon(\sqrt{\nu}\,|z| + \epsilon)}, \qquad z \neq 0.$$

On each half-line the numerator is constant and the positive denominator increases with $|z|$. The supremum on that half-line is therefore the limiting absolute slope at zero. Taking the larger of the two limits gives

$$\mathfrak{L}(a, b, 0, \nu, \epsilon; 0) = \frac{b\epsilon + |a|\sqrt{\nu}}{\epsilon^2}. \tag{61}$$

Now suppose $c = 0$ and $y \neq 0$. On the same side of zero as the reference, the secant reduces to

$$\frac{F(z) - F(y)}{z - y} = \frac{b\epsilon - a\sqrt{\nu}\,\operatorname{sgn}(y)}{(\sqrt{\nu}\,|z| + \epsilon)(\sqrt{\nu}\,|y| + \epsilon)}, \qquad zy \geq 0, \quad z \neq y.$$

The absolute value is largest at $z = 0$, since the numerator is constant and the denominator increases with $|z|$. For inputs on the opposite side, write $z = -\operatorname{sgn}(y)u$, where $u \geq 0$ is the input's distance from zero. Subtracting $F(y)$ and dividing by $z - y = -\operatorname{sgn}(y)(u + |y|)$ gives the rational secant $\mathsf{n}_0(u)/\mathsf{d}_0(u)$ with

$$\begin{aligned} \mathsf{n}_0(u) &= b\big\{\epsilon(u + |y|) + 2\sqrt{\nu}\,|y|u\big\} + a\sqrt{\nu}\,\operatorname{sgn}(y)(u - |y|), \\ \mathsf{d}_0(u) &= (u + |y|)(\sqrt{\nu}\,u + \epsilon)(\sqrt{\nu}\,|y| + \epsilon). \end{aligned}$$

Its value at $u = 0$ is the secant to $z = 0$, which already captures the maximum on the reference's half-line. It therefore suffices to maximize $|\mathsf{n}_0|/\mathsf{d}_0$ over $u \geq 0$.

The denominator d_0 is positive on this domain, so the quotient is continuous. The quotient tends to zero as $u \to \infty$ and is not identically zero: if $\mathsf{n}_0(0) = 0$, the coefficient of u in n_0 is $2b(\epsilon + \sqrt{\nu}\,|y|) > 0$. Thus its absolute value has a positive maximum on a finite interval and hence on $[0, \infty)$. The maximum occurs at $u = 0$ or at a positive stationary point. Since n_0 and d_0 have degrees at most one and two, respectively, the stationary polynomial has degree at most two. It is nonzero because the quotient is nonconstant. Consequently

$$\begin{aligned} \mathcal{Z}_0 &= \{0\} \cup \{u > 0 : \mathsf{n}_0'(u)\mathsf{d}_0(u) - \mathsf{n}_0(u)\mathsf{d}_0'(u) = 0\}, \\ \mathfrak{L}(a, b, 0, \nu, \epsilon; y) &= \max_{u \in \mathcal{Z}_0} \frac{|\mathsf{n}_0(u)|}{\mathsf{d}_0(u)}. \end{aligned} \tag{62}$$

For this case, solve the displayed polynomial, retain its positive real roots, and compare the absolute quotient at those roots and at $u = 0$. Together with Eqs. (60) and (61), this gives a finite value of $\mathfrak{L}$ for every admissible scalar parameter choice.

Return to Adam's coordinate j, whose scalar reference is $y = g_j^{\mathrm{V}}$ and previous second-moment contribution is $c = c_j = \beta_2 q_j/(1-\beta_2^k)$. Insert these and the remaining state parameters from the substitution following Eq. (54) into the applicable case of Eqs. (60)–(62). Scaling its finite value by $\eta/(1-\beta_1^k)$ gives $\lambda_{\mathrm{A},j}$; the largest coordinate value gives Λ_{A}. Both are determined by the fixed state, hyperparameters, and reference gradient.

For any input x, the scalar secant bound and the identity $\Phi_{\mathrm{A}}(x)_j = -\eta F(x_j)/(1-\beta_1^k)$ now give

$$|\Phi_{\mathrm{A}}(x)_j - \Phi_{\mathrm{A}}(g^{\mathrm{V}})_j| \le \lambda_{\mathrm{A},j}|x_j - g_j^{\mathrm{V}}|.$$

We examine the size of Adam's coefficient bounds during training (Figure 10) and find that they remain moderate, so small gradient errors also give small update-error bounds.

This also holds when $x_j = g_j^{\mathrm{V}}$, since both sides are zero. Squaring each coordinate inequality and summing yields

$$\|\Phi_{\mathrm{A}}(x) - \Phi_{\mathrm{A}}(g^{\mathrm{V}})\|^2 \le \sum_j \lambda_{\mathrm{A},j}^2 (x_j - g_j^{\mathrm{V}})^2 \le \Lambda_{\mathrm{A}}^2 \|x - g^{\mathrm{V}}\|^2.$$

Taking square roots gives the bound for every input x; setting $x = \widehat{g}$, the gradient of STOPD, proves Eq. (55). For the mean-square bound, take conditional expectations in the coordinate-weighted inequality at this input. The coefficients are fixed under the conditioning on $(\mathcal{H}_k, \mathcal{Y}_k)$, so its right-hand side becomes $\sum_j \lambda_{\mathrm{A},j}^2 \mathcal{E}_j$. Equation (57) and $\sum_j \mathcal{V}_j = \mathcal{V}$ give

$$\sum_j \mathcal{E}_j = \frac{\mathcal{V}}{C^2} + \rho^2 \|g^{\mathrm{V}}\|^2.$$

Bounding each $\lambda_{\mathrm{A},j}^2$ by Λ_{A}^2 proves Eq. (58). □

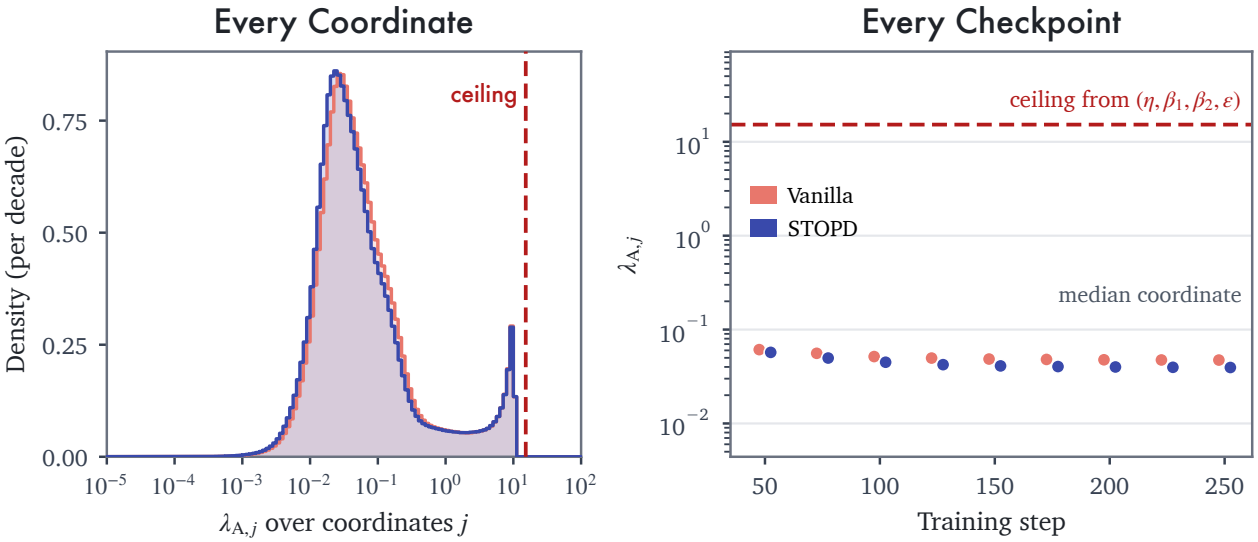


Figure 10 | Upper bounds on Adam's coefficients $\lambda_{\mathrm{A},j}$: distribution across coordinates and checkpoints (left), the median coordinate at each step averaged over the five settings and three seeds (right), and the theoretical ceiling (dashed).

B.3. Muon

Let $x \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ be a matrix gradient input. Fix the previous momentum $\boldsymbol{M}$, the momentum parameter $\beta \in [0, 1)$, and the stabilization parameter $\epsilon_{\text{M}} > 0$. We use Frobenius norms except for the operator norm $\|\cdot\|_{\text{op}}$, and restrict g^{V}, $\widehat{g}$, and $\mathcal{V}$ to this block. Muon forms the new momentum $\boldsymbol{M}^{+}(x) = \beta\boldsymbol{M} + (1-\beta)x$ and then the Nesterov input

$$\boldsymbol{R}(x) = (1-\beta)x + \beta\boldsymbol{M}^{+}(x) = \beta^2\boldsymbol{M} + (1-\beta^2)x.$$

We normalize the Nesterov input by clamping its Frobenius norm below at ϵ_{M}:

$$\boldsymbol{X}_0(x) = \frac{\boldsymbol{R}(x)}{\max\{\|\boldsymbol{R}(x)\|_F, \epsilon_{\text{M}}\}}.$$

Muon then applies $J \geq 0$ polynomial iterations, with J an integer [Jordan et al., 2024]:

$$\boldsymbol{X}_{j+1} = \omega_1\boldsymbol{X}_j + \omega_3\boldsymbol{X}_j\boldsymbol{X}_j^{\top}\boldsymbol{X}_j + \omega_5(\boldsymbol{X}_j\boldsymbol{X}_j^{\top})^2\boldsymbol{X}_j. \tag{63}$$

Here $\omega_1, \omega_3, \omega_5 \in \mathbb{R}$ are the optimizer's fixed polynomial coefficients. Let $\mathcal{P}_J$ denote the composition of these matrix iterations, with $\mathcal{P}_0(X) = X$. Let $\gamma_{\text{shape}} > 0$ be the optimizer's fixed scaling factor for this block, determined by its original dimensions and the chosen scaling convention. Its parameter increment is

$$\Phi_{\text{M}}(x) = -\eta\gamma_{\text{shape}}\mathcal{P}_J(\boldsymbol{X}_0(x)).$$

The update comparison uses the reference Nesterov matrix and its normalization,

$$\boldsymbol{B} = \boldsymbol{R}(g^{\text{V}}), \qquad b = \|\boldsymbol{B}\|_F, \qquad Y = \boldsymbol{X}_0(g^{\text{V}}) = \frac{\boldsymbol{B}}{\max\{b, \epsilon_{\text{M}}\}}.$$

The effect of normalization on the distance from this reference is controlled by

$$\kappa_b = \frac{2}{\max\{b, \epsilon_{\text{M}}\} + \epsilon_{\text{M}}}. \tag{64}$$

For the polynomial iterations, write the scalar one-step polynomial as $\varphi(s) = \omega_1 s + \omega_3 s^3 + \omega_5 s^5$ and its compositions as $f_0(s) = s$, $f_{j+1} = \varphi \circ f_j$. Their scalar sensitivity is the divided difference

$$\mathrm{D}_J(s,t) = \begin{cases} \dfrac{f_J(s) - f_J(t)}{s - t}, & s \neq t, \\ f_J'(t), & s = t. \end{cases} \tag{65}$$

Since f_J is a polynomial, D_J is a polynomial in (s,t) including on the diagonal, and $\mathrm{D}_0 = 1$. To relate these scalar polynomials to the rectangular matrix iterations [Andersson et al., 2016], use the symmetric dilation

$$\mathcal{D}(Z) = \begin{pmatrix} 0 & Z \\ Z^{\top} & 0 \end{pmatrix}, \qquad \mathcal{T}_Y = \mathrm{spec}(\mathcal{D}(Y)).$$

The dilation eigenvalues are the signed nonzero singular values, together with the remaining zeros. Since normalized inputs have Frobenius norm at most one, their dilation spectra lie in $[-1, 1]$. We combine κ_b with divided differences over this interval and the fixed reference spectrum $\mathcal{T}_Y$.

Proposition B.2 (Update error with Muon). *For the current batch, let $\widehat{g}$ and g^{V} denote the gradients of STOPD and Vanilla, respectively, restricted to a matrix parameter block. The Muon updates defined by Φ_{M} above, from the same parameter values and previous momentum $\boldsymbol{M}$, satisfy*

$$\boxed{\|\Phi_{\text{M}}(\widehat{g}) - \Phi_{\text{M}}(g^{\text{V}})\|_F \leq \Lambda_{\text{M}}\|\widehat{g} - g^{\text{V}}\|_F}. \tag{66}$$

Here the coefficient is

$$\Lambda_{\mathrm{M}} = \eta\gamma_{\mathrm{shape}}(1-\beta^2)\kappa_b \max_{\substack{|s|\le 1\\ t\in\mathcal{T}_Y}} |\mathsf{D}_J(s,t)|. \tag{67}$$

This coefficient is finite and fixed conditional on the training history and complete batch. The same pointwise bound holds with $\widehat{g}$ replaced by any matrix gradient input x of the same shape. Averaging over the current cutoffs of STOPD bounds the conditional mean-square update error:

$$\mathbb{E}_S\|\Phi_{\mathrm{M}}(\widehat{g}) - \Phi_{\mathrm{M}}(g^{\mathrm{V}})\|_F^2 \le \Lambda_{\mathrm{M}}^2 \left\{\frac{\mathcal{V}}{C^2} + \rho^2\|g^{\mathrm{V}}\|_F^2\right\}. \tag{68}$$

For a block of the globally clipped update in Eq. (50), evaluate Eq. (67) with $\boldsymbol{B} = \boldsymbol{R}(y)$, where y is the corresponding block of the globally clipped full Vanilla gradient.

Proof. We track the gradient difference through the Nesterov input, normalization, and polynomial iterations, then combine their factors to obtain the update-error coefficient. Fix a matrix gradient input x. The shared previous momentum cancels between $\boldsymbol{R}(x)$ and $\boldsymbol{B} = \boldsymbol{R}(g^{\mathrm{V}})$:

$$\boldsymbol{R}(x) - \boldsymbol{B} = (1-\beta^2)(x - g^{\mathrm{V}}).$$

Write $r = \|\boldsymbol{R}(x)\|_F$ and $b = \|\boldsymbol{B}\|_F$ for their Frobenius norms. Normalization maps them to $\boldsymbol{X}_0(x)$ and $Y = \boldsymbol{X}_0(g^{\mathrm{V}})$, with respective Frobenius norms $a = r/\max\{r, \epsilon_{\mathrm{M}}\}$ and $c = b/\max\{b, \epsilon_{\mathrm{M}}\}$. Suppose first that $r, b > 0$ and $r \ne b$, and let $\cos\vartheta = \langle \boldsymbol{R}(x), \boldsymbol{B}\rangle_F/(rb)$. Expanding the squared normalization-distance ratio $\|\boldsymbol{X}_0(x) - Y\|_F^2/\|\boldsymbol{R}(x) - \boldsymbol{B}\|_F^2$ gives

$$\frac{a^2 + c^2 - 2ac\cos\vartheta}{r^2 + b^2 - 2rb\cos\vartheta}.$$

Its denominator is at least $(r-b)^2 > 0$. Both numerator and denominator are affine in $\cos\vartheta \in [-1, 1]$, so differentiation gives a constant sign and the maximum occurs at an endpoint. For $\cos\vartheta = -1$ and $\cos\vartheta = 1$, the square roots of the ratio are, respectively,

$$\frac{a+c}{r+b} \quad \text{and} \quad \frac{|a-c|}{|r-b|}.$$

The first is at least the second. Indeed, when $r > b$, we have $a \ge c$ and $a/r \le c/b$, hence $(a-c)(r+b) \le (a+c)(r-b)$. Interchanging the radii proves the same comparison when $r < b$.

The cases excluded from this ratio calculation give the same factor. When $r = b > 0$, both matrices are scaled by $1/\max\{r, \epsilon_{\mathrm{M}}\} = (a+c)/(r+b)$, including when the matrices coincide. If exactly one radius is zero, the distance ratio is again $(a+c)/(r+b)$. If both vanish, both distances vanish. Define $q_b(r) = (a+c)/(r+b)$ when $r + b > 0$, and $q_0(0) = 1/\epsilon_{\mathrm{M}}$. Then every input satisfies

$$\|\boldsymbol{X}_0(x) - Y\|_F \le q_b(r)\|\boldsymbol{R}(x) - \boldsymbol{B}\|_F \le \kappa_b\|\boldsymbol{R}(x) - \boldsymbol{B}\|_F. \tag{69}$$

To obtain the second inequality, first take $b \le \epsilon_{\mathrm{M}}$. Each normalized radius is at most its original radius divided by ϵ_{M}, so $q_b(r) \le 1/\epsilon_{\mathrm{M}} = \kappa_b$. For $b > \epsilon_{\mathrm{M}}$,

$$q_b(r) = \begin{cases} \dfrac{r/\epsilon_{\mathrm{M}} + 1}{r+b}, & 0 \le r \le \epsilon_{\mathrm{M}}, \\[2ex] \dfrac{2}{r+b}, & r \ge \epsilon_{\mathrm{M}}. \end{cases}$$

The first expression has derivative $(b/\epsilon_{\mathrm{M}} - 1)/(r+b)^2 > 0$; the second decreases with r. Their common maximum at $r = \epsilon_{\mathrm{M}}$ is $2/(b + \epsilon_{\mathrm{M}}) = \kappa_b$. For every $b > 0$, choosing $\boldsymbol{R}(x) = -\epsilon_{\mathrm{M}}\boldsymbol{B}/b$ attains the factor κ_b.

When $b = 0$, any input with $\|\boldsymbol{R}(x)\|_F = \epsilon_{\mathrm{M}}$ attains it. Thus κ_b is the exact distance-amplification factor of normalization about $\boldsymbol{B}$.

For a matrix X of the same shape as the normalized reference $Y = \boldsymbol{X}_0(g^{\mathrm{V}})$, the symmetric dilation $\mathcal{D}$ relates the scalar polynomial φ and its compositions f_J to the matrix iterations. Block multiplication and iteration give

$$\varphi(\mathcal{D}(X)) = \mathcal{D}(\mathcal{P}_1(X)), \qquad f_J(\mathcal{D}(X)) = \mathcal{D}(\mathcal{P}_J(X)).$$

The second identity includes $J = 0$, since both compositions are then the identity map.

For $\|X\|_{\mathrm{op}} \le 1$, choose eigenvalue–eigenvector pairs (s_u, p_u) of $\mathcal{D}(X)$ and (t_v, q_v) of $\mathcal{D}(Y)$, with both eigenvector bases orthonormal. The dilation eigenvalues are signed singular values (and zeros), so the input eigenvalues satisfy $|s_u| \le 1$, and the reference eigenvalues lie in $\mathcal{T}_Y = \mathrm{spec}(\mathcal{D}(Y))$. The divided difference D_J of f_J relates the corresponding entries in these two bases:

$$\begin{aligned} p_u^\top \{f_J(\mathcal{D}(X)) - f_J(\mathcal{D}(Y))\} q_v &= \{f_J(s_u) - f_J(t_v)\} p_u^\top q_v \\ &= \mathsf{D}_J(s_u, t_v)\, p_u^\top \{\mathcal{D}(X) - \mathcal{D}(Y)\} q_v. \end{aligned}$$

When $s_u = t_v$, both sides of the second equality are zero. The sum of squared entries in these orthonormal bases is the squared Frobenius norm. Bound every divided difference by $\max_{|s|\le 1,\, t\in\mathcal{T}_Y} |\mathsf{D}_J(s,t)|$, sum the squares, and take square roots. The dilation identity above and $\|\mathcal{D}(Z)\|_F = \sqrt{2}\|Z\|_F$ then give

$$\|\mathcal{P}_J(X) - \mathcal{P}_J(Y)\|_F \le \max_{\substack{|s|\le 1 \\ t\in\mathcal{T}_Y}} |\mathsf{D}_J(s,t)|\, \|X - Y\|_F.$$

For the actual normalized input $X = \boldsymbol{X}_0(x)$, $\|X\|_{\mathrm{op}} \le \|X\|_F \le 1$, so this polynomial bound applies. Its coefficient is finite: $\mathcal{T}_Y$ is finite, and each $\mathsf{D}_J(\cdot, t)$ is a polynomial on the compact interval $[-1, 1]$. To compute Λ_{M} from the fixed state, form $\boldsymbol{B}$, b, Y, and f_J. For each reference eigenvalue $t \in \mathcal{T}_Y$, maximize $|\mathsf{D}_J(s,t)|$ over $s \in [-1, 1]$ by evaluating the endpoints and the stationary points of $\mathsf{D}_J(\cdot, t)$; a constant polynomial needs only an endpoint. Take the largest value over t and multiply by the displayed factors.

The parameter increments are $-\eta\gamma_{\mathrm{shape}}$ times the polynomial outputs, so the two bounds give

$$\begin{aligned} \|\Phi_{\mathrm{M}}(x) - \Phi_{\mathrm{M}}(g^{\mathrm{V}})\|_F &\le \eta\gamma_{\mathrm{shape}}\kappa_b \max_{\substack{|s|\le 1 \\ t\in\mathcal{T}_Y}} |\mathsf{D}_J(s,t)|\, \|\boldsymbol{R}(x) - \boldsymbol{B}\|_F \\ &= \Lambda_{\mathrm{M}} \|x - g^{\mathrm{V}}\|_F. \end{aligned}$$

The last step uses the Nesterov-input difference established at the start of the proof and the maximum defining Λ_{M}. This gives the bound for every matrix input x; setting $x = \widehat{g}$, the gradient of STOPD, proves Eq. (66).

The coefficient is fixed under conditioning on history and the complete batch. Squaring at $x = \widehat{g}$ and averaging over the current cutoffs gives

$$\mathbb{E}_S \|\Phi_{\mathrm{M}}(\widehat{g}) - \Phi_{\mathrm{M}}(g^{\mathrm{V}})\|_F^2 \le \Lambda_{\mathrm{M}}^2 \mathbb{E}_S \|\widehat{g} - g^{\mathrm{V}}\|_F^2.$$

Substituting the conditional gradient-error decomposition Eq. (25) for this block proves Eq. (68). □

We examine the size of Muon's coefficient bounds during training (Figure 11) and find that they remain moderate, so small gradient errors also give small update-error bounds.

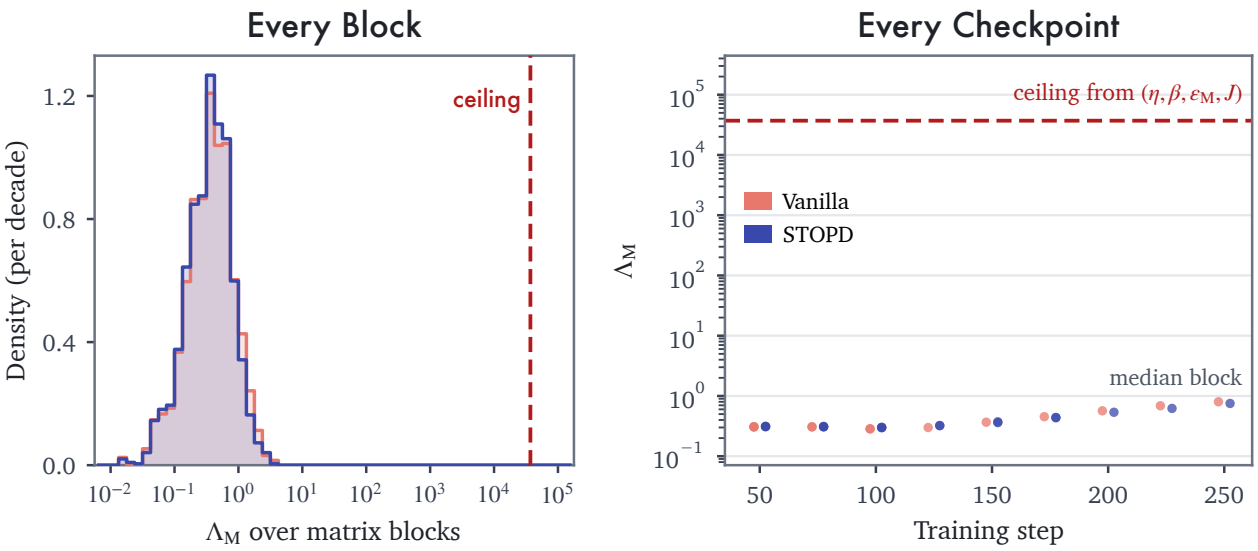


Figure 11 | Muon's coefficients Λ_M: distribution across blocks and checkpoints (left), the median block at each checkpoint (right), and the theoretical ceiling (dashed).

C. Token and Time Savings

We first derive the expected token savings, then analyze how these savings translate into reduced generation time.

C.1. Token savings

For a complete rollout of length T, random truncation skips token t with probability $1 - p_t$. The expected number of tokens saved is therefore

$$D(T) = \sum_{t=1}^{T} (1 - p_t).$$

Tokens within the same block have the same inclusion probability. Position t belongs to block $b = \lfloor (t-1)/K \rfloor$, where $b = 0$ denotes the first block. Under the stopping rule in Eq. (1), its inclusion probability is

$$p_t = \frac{L - K\lfloor (t-1)/K \rfloor}{L} = 1 - \frac{bK}{L} > 0. \tag{70}$$

Every token in block b is therefore skipped with probability bK/L. Write $T = mK + r$, where m is the number of complete blocks and $0 \le r < K$ is the number of tokens in the next block. Each complete block contributes $K \cdot bK/L$ expected saved tokens, while the remaining r tokens contribute $r \cdot mK/L$. Adding these contributions gives

$$D(T) = \frac{K}{L}\left(K \sum_{b=0}^{m-1} b + mr\right) = \frac{K}{L}\left\{\frac{Km(m-1)}{2} + mr\right\}. \tag{71}$$

Substituting $m = (T - r)/K$ makes the dependence on rollout length explicit:

$$D(T) = \frac{T^2 - KT + r(K - r)}{2L}.$$

For $K \ll T$, the terms involving K are small compared with T^2, giving

$$D(T) \approx \frac{T^2}{2L}, \qquad \frac{D(T)}{T} \approx \frac{T}{2L}.$$

Thus longer rollouts save a larger fraction of tokens: about 25% for $T = L/2$ and nearly 50% for $T = L$. At the max length, when K divides L, the exact saving is $D(L) = (L - K)/2$. Rollouts ending within the first block ($T \le K$) save no tokens.

Subtracting the expected saving from the complete length gives the expected generated length of rollout i:

$$\mathbb{E}_S M_i = T_i - D(T_i).$$

Here, $\mathbb{E}_S$ averages over the sampled stopping positions for a given complete rollout length T_i. In particular, a rollout reaching the max length retains $(L + K)/2$ tokens on average when K divides L.

For a batch of B rollouts, Vanilla OPD generates $N = \sum_{i=1}^{B} T_i$ tokens. Random truncation never generates more tokens than this. Summing the per-rollout expectations gives

$$\sum_i M_i \le N \quad \text{for every set of stopping positions,} \qquad \mathbb{E}_S \sum_i M_i = N - \sum_i D(T_i). \tag{72}$$

C.2. Time savings

We model generation time using the number of decoding rounds and the context lengths processed in those rounds. During autoregressive decoding, the key–value (KV) cache stores attention keys and values for the prompt and previously generated tokens. Generating each new token requires a forward pass, whose attention reads this growing cached context. Random truncation avoids forward-pass computation for skipped tokens, reduces KV-cache reads, and limits the growth of each rollout's cache. This model explains when random truncation can save more time than token savings alone would predict. Let $\mathcal{R}_r$ be the set of active rollouts in decoding round r, each generating one token. Over R rounds, we approximate the time per round and the total decoding time as

$$\tau_r \approx \alpha + \beta \sum_{j \in \mathcal{R}_r} c_{j,r}, \qquad t_{\text{decode}} \approx \alpha R + \beta \mathcal{A}, \qquad \mathcal{A} = \sum_{r=1}^{R} \sum_{j \in \mathcal{R}_r} c_{j,r}, \tag{73}$$

where $c_{j,r}$ is the context length of rollout j in round r and $\alpha \ge 0$ is the fixed time cost of each round. The coefficient $\beta \ge 0$ models the additional time per context token processed. We call the cumulative sum $\mathcal{A}$ the KV area: it measures the total cached context processed across decoding rounds. For a fixed model, it serves as a proxy for attention work and KV-cache reads.

KV area. For rollout i with prompt length T_i^{p}, the context before generating token $t \ge 1$ contains T_i^{p} prompt tokens and $t - 1$ earlier rollout tokens. Adding these context lengths over the generated tokens gives

$$\mathcal{A}_i = \sum_{t=1}^{M_i} (T_i^{\mathrm{p}} + t - 1) = \sum_{t=1}^{T_i} \mathbf{1}\{S_i \ge t\}(T_i^{\mathrm{p}} + t - 1), \qquad \mathcal{A} = \sum_{i=1}^{B} \mathcal{A}_i. \tag{74}$$

Using $\mathbb{E}_S[\mathbf{1}\{S_i \ge t\}] = p_t$ and $\sum_{t=1}^{T_i} p_t = T_i - D(T_i)$ gives

$$\mathbb{E}_S[\mathcal{A}_i] = \sum_{t=1}^{T_i} p_t (T_i^{\mathrm{p}} + t - 1) = T_i^{\mathrm{p}} \{T_i - D(T_i)\} + \sum_{t=1}^{T_i} (t-1) p_t, \tag{75}$$

$$\mathcal{A}_i^{\mathrm{V}} = \sum_{t=1}^{T_i} (T_i^{\mathrm{p}} + t - 1) = T_i^{\mathrm{p}} T_i + \frac{T_i(T_i - 1)}{2}, \qquad \mathcal{A}_i^{\mathrm{V}} - \mathbb{E}_S[\mathcal{A}_i] = \sum_{t=1}^{T_i} (1 - p_t)(T_i^{\mathrm{p}} + t - 1). \tag{76}$$

Truncation is more likely to skip later tokens, each of which would require processing a longer context.

To account for block boundaries, let r_t count the positions before token t within its block. Eq. (2) then gives

$$p_t = 1 - \frac{t - 1 - r_t}{L}, \qquad r_t = (t-1) - K \left\lfloor \frac{t-1}{K} \right\rfloor, \qquad 0 \le r_t < K. \tag{77}$$

For $K \ll T_i$, summing over positions yields

$$\begin{aligned}
\mathbb{E}_S[M_i] &= \sum_{t=1}^{T_i} p_t = T_i - \frac{T_i(T_i-1)}{2L} + \frac{1}{L}\sum_{t=1}^{T_i} r_t \\
&= T_i - \frac{T_i^2}{2L} + O\left(\frac{KT_i}{L}\right), \\
\sum_{t=1}^{T_i}(t-1)p_t &= \frac{T_i(T_i-1)}{2} - \frac{T_i(T_i-1)(2T_i-1)}{6L} + \frac{1}{L}\sum_{t=1}^{T_i}(t-1)r_t \\
&= \frac{T_i^2}{2} - \frac{T_i^3}{3L} + O\left(T_i + \frac{KT_i^2}{L}\right).
\end{aligned} \tag{78}$$

Random truncation therefore saves the following amounts of tokens and KV area from the generated rollout context:

$$T_i - \mathbb{E}_S[M_i] \approx \frac{T_i^2}{2L}, \qquad \frac{T_i(T_i-1)}{2} - \sum_{t=1}^{T_i}(t-1)p_t \approx \frac{T_i^3}{3L}. \tag{79}$$

The token saving counts positions whose forward-pass computation is avoided. The KV-area saving measures the cached rollout context that attention no longer needs to read at those positions.

Each generated token also uses the T_i^{p} prompt tokens. Adding this contribution gives

$$\mathbb{E}_S[\mathcal{A}_i] \approx T_i^{\mathrm{p}}\left(T_i - \frac{T_i^2}{2L}\right) + \frac{T_i^2}{2} - \frac{T_i^3}{3L}, \qquad \mathcal{A}_i^{\mathrm{V}} - \mathbb{E}_S[\mathcal{A}_i] \approx \frac{T_i^{\mathrm{p}}T_i^2}{2L} + \frac{T_i^3}{3L}. \tag{80}$$

When prompts are much shorter than rollouts, we can neglect their contribution to the KV area. Writing $u_i = T_i/L$ for the rollout length as a fraction of the max length gives

$$\mathbb{E}_S[M_i] \approx T_i\left(1 - \frac{u_i}{2}\right), \qquad \mathbb{E}_S[\mathcal{A}_i] \approx \frac{T_i^2}{2}\left(1 - \frac{2u_i}{3}\right). \tag{81}$$

Dividing Vanilla OPD's token count and KV area by the corresponding expectations under random truncation gives

$$\frac{T_i}{\mathbb{E}_S[M_i]} \approx \frac{1}{1-u_i/2}, \qquad \frac{\mathcal{A}_i^{\mathrm{V}}}{\mathbb{E}_S[\mathcal{A}_i]} \approx \frac{1}{1-2u_i/3}. \tag{82}$$

Dividing the area ratio by the token-count ratio shows how much the average context per generated token decreases:

$$\frac{\mathcal{A}_i^{\mathrm{V}}/T_i}{\mathbb{E}_S[\mathcal{A}_i]/\mathbb{E}_S[M_i]} \approx \frac{1-u_i/2}{1-2u_i/3}. \tag{83}$$

For $T_i = L$, the expected generated length is

$$\mathbb{E}_S[M_i] = \frac{L+K}{2} \tag{84}$$

when K divides L. Under the same small-block and short-prompt approximations, a rollout of length $T_i = L/2$ saves about 25% of its tokens and 33% of its KV area in expectation. At $T_i = L$, these savings increase to about 50% and 67%, respectively. Random truncation thus reduces the context processed by a larger fraction than the token count.

Effect of scheduling. When processing long contexts dominates the cost in Eq. (73), decoding time is approximately proportional to KV area, so time savings can exceed the fraction of tokens saved.

Scheduling also affects the speedup by determining the number of decoding rounds R. We examine two cases, with batch areas $\mathcal{A} = \sum_i \mathcal{A}_i$ and $\mathcal{A}^{\mathrm{V}} = \sum_i \mathcal{A}_i^{\mathrm{V}}$.

First, suppose new rollouts can start as others finish, keeping about Q context tokens active in the KV cache of both methods during most rounds. We ignore the brief periods at the beginning and end when fewer rollouts are active:

$$\sum_{j \in \mathcal{R}_r} c_{j,r} \approx Q. \tag{85}$$

Each round contributes about Q to the KV area, so

$$\mathcal{A} \approx QR, \qquad R \approx \frac{\mathcal{A}}{Q}, \qquad t_{\mathrm{decode}} \approx \left(\frac{\alpha}{Q} + \beta\right)\mathcal{A}. \tag{86}$$

Dividing the generated-token count $N^{\mathrm{S}} = \sum_i M_i$ by decoding time gives the throughput:

$$\frac{N^{\mathrm{S}}}{t_{\mathrm{decode}}} \approx \frac{1}{\alpha/Q + \beta}\frac{N^{\mathrm{S}}}{\mathcal{A}}. \tag{87}$$

Let t_{V} denote Vanilla OPD's decoding time. The predicted speedup under this scheduling regime is

$$\frac{t_{\mathrm{V}}}{\mathbb{E}_S[t_{\mathrm{decode}}]} \approx \frac{\mathcal{A}^{\mathrm{V}}}{\mathbb{E}_S[\mathcal{A}]} = \frac{N}{\mathbb{E}_S[N^{\mathrm{S}}]}\frac{\mathcal{A}^{\mathrm{V}}/N}{\mathbb{E}_S[\mathcal{A}]/\mathbb{E}_S[N^{\mathrm{S}}]}. \tag{88}$$

Equation (88) separates the predicted speedup into the reduction in token count and the reduction in mean context per token. The first factor reflects fewer tokens and hence less forward-pass computation. The second captures the throughput gain from shorter cached contexts: at the same active cache occupancy, more rollouts can generate concurrently. When the prompt contribution is neglected, both factors are at least one.

Second, suppose all rollouts start together and generate one token per round without pauses. No new rollouts enter the batch, so generation ends when the longest rollout finishes:

$$R = \max_i M_i, \qquad t_{\mathrm{decode}} \approx \alpha \max_i M_i + \beta\mathcal{A}. \tag{89}$$

Consider n rollouts whose complete lengths equal L. Ignoring block rounding, let

$$U_1, \ldots, U_n \overset{\mathrm{iid}}{\sim} \mathrm{Uniform}(0, L), \qquad Z = \max_i U_i. \tag{90}$$

For $0 \le z \le L$, the largest stopping position is at most z only when all n stopping positions are at most z. Each has probability z/L of doing so. Independence lets us multiply these probabilities; integrating the resulting tail probability gives the mean:

$$\Pr(Z \le z) = \left(\frac{z}{L}\right)^n, \qquad \mathbb{E}[Z] = \int_0^L \left[1 - \left(\frac{z}{L}\right)^n\right] \mathrm{d}z = \frac{nL}{n+1}. \tag{91}$$

The expected gap between the max length and the largest stopping position is therefore

$$L - \mathbb{E}[Z] = \frac{L}{n+1}. \tag{92}$$

Rounding U_i up to a block boundary without exceeding the max length L gives

$$U_i \le S_i \le \min\{U_i + K, L\}, \tag{93}$$

Taking the maximum over rollouts and then the expectation gives

$$\frac{nL}{n+1} \le \mathbb{E}_S\left[\max_i S_i\right] \le \min\left\{L,\ \frac{nL}{n+1} + K\right\}. \tag{94}$$

Random truncation can accelerate generation even when the longest retained rollout remains close to L. It still removes many tokens across the batch, avoiding their forward-pass computation and KV-cache reads. In the time model, this reduces $\beta\mathcal{A}$ even when αR changes little. The greater the share of decoding time spent processing cached contexts, the larger the resulting speedup.

Practical implications. A larger batch can improve generation throughput while reducing truncation noise. When the scheduler admits waiting rollouts as others finish, it can reuse the KV-cache space freed by random truncation. A larger batch supplies more waiting rollouts, helping sustain concurrency and cache occupancy. The student update then combines gradient contributions from these same rollouts, averaging over more independent stopping positions and reducing the gradient variance caused by random truncation (Appendix A.5).

D. Accuracy by Step

Table 2 lists the accuracy behind Figures 2 and 4 at every evaluated checkpoint, and Tables 3 and 4 the accuracy behind Figures 8 and 12.

Table 2 | Accuracy (%) every 25 steps, mean ± standard deviation, from the base model and the shared step-25 Vanilla warmup to 250 steps.

			Accuracy (%) at step									
Setting	**Method**	**Base**	**25**	**50**	**75**	**100**	**125**	**150**	**175**	**200**	**225**	**250**
Main results: Qwen3-1.7B-Base ← Qwen3-4B-Instruct-2507												
Math	Vanilla	11.7	$23.4_{\pm 0.8}$	$24.1_{\pm 0.0}$	$27.9_{\pm 0.7}$	$26.9_{\pm 0.2}$	$28.4_{\pm 0.6}$	$28.5_{\pm 0.7}$	$29.3_{\pm 0.4}$	$28.1_{\pm 1.8}$	$30.0_{\pm 0.4}$	$30.1_{\pm 0.5}$
	STOPD			$24.4_{\pm 0.9}$	$27.3_{\pm 0.7}$	$27.5_{\pm 0.5}$	$27.7_{\pm 0.6}$	$29.0_{\pm 0.5}$	$29.1_{\pm 0.1}$	$27.9_{\pm 1.4}$	$28.5_{\pm 0.6}$	$29.4_{\pm 0.6}$
Code	Vanilla	21.8	$27.6_{\pm 0.2}$	$26.4_{\pm 0.2}$	$25.5_{\pm 0.3}$	$26.5_{\pm 0.2}$	$27.2_{\pm 0.3}$	$27.6_{\pm 0.2}$	$27.6_{\pm 0.3}$	$28.1_{\pm 0.3}$	$28.5_{\pm 0.1}$	$28.5_{\pm 0.1}$
	STOPD			$26.3_{\pm 0.3}$	$25.7_{\pm 0.4}$	$25.8_{\pm 0.3}$	$26.9_{\pm 0.3}$	$27.4_{\pm 0.3}$	$27.2_{\pm 0.1}$	$27.5_{\pm 0.6}$	$27.8_{\pm 0.4}$	$28.1_{\pm 0.4}$
Instruction following	Vanilla	15.6	$24.6_{\pm 0.1}$	$34.8_{\pm 0.5}$	$44.4_{\pm 0.7}$	$49.9_{\pm 1.0}$	$52.2_{\pm 0.2}$	$54.3_{\pm 0.2}$	$57.0_{\pm 0.5}$	$58.6_{\pm 1.1}$	$59.9_{\pm 1.4}$	$59.8_{\pm 1.3}$
	STOPD			$35.0_{\pm 0.7}$	$44.3_{\pm 0.5}$	$50.6_{\pm 0.2}$	$51.3_{\pm 0.5}$	$53.6_{\pm 0.3}$	$56.2_{\pm 0.3}$	$59.0_{\pm 0.5}$	$59.1_{\pm 1.2}$	$60.1_{\pm 0.9}$
Scaling: mathematics, Qwen3-30B-A3B-Instruct-2507 teacher												
4B	Vanilla	15.2	$34.9_{\pm 0.8}$	$34.7_{\pm 0.7}$	$41.1_{\pm 0.7}$	$44.0_{\pm 0.6}$	$44.2_{\pm 0.2}$	$45.0_{\pm 0.5}$	$45.0_{\pm 0.4}$	$46.2_{\pm 0.9}$	$46.3_{\pm 0.3}$	$46.2_{\pm 0.2}$
	STOPD			$34.7_{\pm 0.6}$	$40.1_{\pm 0.6}$	$43.5_{\pm 0.2}$	$43.7_{\pm 0.3}$	$44.9_{\pm 0.7}$	$44.9_{\pm 0.5}$	$45.5_{\pm 0.6}$	$46.3_{\pm 0.7}$	$45.3_{\pm 0.4}$
8B	Vanilla	23.5	$38.0_{\pm 0.3}$	$39.3_{\pm 1.3}$	$45.8_{\pm 0.8}$	$47.1_{\pm 0.2}$	$48.3_{\pm 0.4}$	$48.3_{\pm 0.2}$	$49.3_{\pm 0.4}$	$50.1_{\pm 0.1}$	$50.2_{\pm 0.3}$	$50.1_{\pm 0.1}$
	STOPD			$38.7_{\pm 0.9}$	$44.6_{\pm 0.1}$	$47.3_{\pm 0.8}$	$47.2_{\pm 0.9}$	$48.8_{\pm 0.7}$	$48.6_{\pm 0.4}$	$49.9_{\pm 0.8}$	$49.1_{\pm 0.6}$	$49.7_{\pm 0.8}$

Table 3 | Accuracy (%) every 50 steps for each block width K in mathematics (1.7B ← 4B), mean ± standard deviation.

			Accuracy (%) at step					
K	**Method**	**Base**	**25**	**50**	**100**	**150**	**200**	**250**
	Vanilla			$24.1_{\pm 0.0}$	$26.9_{\pm 0.2}$	$28.5_{\pm 0.7}$	$28.1_{\pm 1.8}$	$30.1_{\pm 0.5}$
32	STOPD			$24.1_{\pm 0.2}$	$27.1_{\pm 0.5}$	$28.2_{\pm 0.6}$	$29.2_{\pm 0.5}$	$29.8_{\pm 0.7}$
128	STOPD	11.7	$23.4_{\pm 0.8}$	$24.4_{\pm 0.9}$	$27.5_{\pm 0.5}$	$29.0_{\pm 0.5}$	$27.9_{\pm 1.4}$	$29.4_{\pm 0.6}$
512	STOPD			$24.1_{\pm 0.7}$	$27.7_{\pm 0.5}$	$28.8_{\pm 0.2}$	$28.7_{\pm 0.8}$	$29.7_{\pm 0.2}$
2048	STOPD			$23.7_{\pm 0.2}$	$26.9_{\pm 0.3}$	$28.4_{\pm 0.6}$	$28.5_{\pm 1.0}$	$30.0_{\pm 0.2}$

Table 4 | IFEval (%) at $B = 2048$ every 5 steps, mean ± standard deviation, from the base model and the shared step-5 Vanilla warmup.

		IFEval (%) at step									
Method	**Base**	**5**	**10**	**15**	**20**	**25**	**30**	**35**	**40**	**45**	**50**
Vanilla	15.6	$16.2_{\pm 0.1}$	$17.4_{\pm 0.2}$	$18.0_{\pm 0.1}$	$25.3_{\pm 0.8}$	$29.4_{\pm 0.5}$	$30.5_{\pm 0.2}$	$32.9_{\pm 0.5}$	$35.9_{\pm 0.6}$	$38.5_{\pm 0.8}$	$40.5_{\pm 0.7}$
STOPD			$17.4_{\pm 0.2}$	$17.5_{\pm 0.5}$	$25.3_{\pm 0.4}$	$28.6_{\pm 0.3}$	$30.1_{\pm 0.2}$	$32.8_{\pm 0.4}$	$35.4_{\pm 0.5}$	$37.8_{\pm 0.7}$	$40.0_{\pm 0.4}$

E. Additional Time Decomposition

Tables 5, 6, 7, and 8 break down the training step of the 4B and 8B students of Section 5.2, of the optimizers of Section 5.3, of the block widths of Section 5.4, and of instruction following at both batch sizes of Section 5.5 as Table 1 does for the main setting.

Table 5 | Cost of a training step for the 4B and 8B students in mathematics.

				Time per step (s)			
Student	**Method**	**Tokens**	**k tok/s**	**Generation**	**Update**	**Other**	**Total**
4B	Vanilla	10,005	3.4	753	360	95	1209
	STOPD	6,511	4.3	392	219	61	672
	Speedup	1.5×	1.3×	1.9×	1.6×	1.6×	**1.8**×
8B	Vanilla	9,432	5.6	434	303	81	818
	STOPD	6,276	6.5	246	190	54	490
	Speedup	1.5×	1.2×	1.8×	1.6×	1.5×	**1.7**×

Table 6 | Cost of a training step under each optimizer in mathematics.

Qwen3-1.7B-Base ← *Qwen3-4B-Instruct-2507*				Time per step (s)			
Optimizer	**Method**	**Tokens**	**k tok/s**	**Generation**	**Update**	**Other**	**Total**
AdamW	Vanilla	9,516	3.5	697	189	56	941
	STOPD	6,255	4.5	358	119	36	513
	Speedup	1.5×	1.3×	2.0×	1.6×	1.5×	**1.8**×
SGD	Vanilla	9,351	2.9	814	185	49	1049
	STOPD	5,509	4.1	346	111	35	492
	Speedup	1.7×	1.4×	2.4×	1.7×	1.4×	**2.1**×
Muon	Vanilla	10,014	3.3	780	191	52	1023
	STOPD	6,248	4.2	381	126	41	549
	Speedup	1.6×	1.3×	2.0×	1.5×	1.3×	**1.9**×

Table 7 | Cost of a training step for each block width K in mathematics.

Qwen3-1.7B-Base ← *Qwen3-4B-Instruct-2507*				Time per step (s)			
K	Method	Tokens	k tok/s	Generation	Update	Other	Total
	Vanilla	9,516	3.5	697	189	56	941
32	STOPD	6,227	4.4	366	124	41	531
	Speedup	1.5×	1.2×	1.9×	1.5×	1.4×	**1.8×**
128	STOPD	6,255	4.5	358	119	36	513
	Speedup	1.5×	1.3×	2.0×	1.6×	1.5×	**1.8×**
512	STOPD	6,365	4.4	372	124	39	535
	Speedup	1.5×	1.3×	1.9×	1.5×	1.4×	**1.8×**
2048	STOPD	6,792	4.4	397	131	41	569
	Speedup	1.4×	1.3×	1.8×	1.4×	1.4×	**1.7×**

Table 8 | Cost of a training step in instruction following at $B = 256$ and $B = 2048$.

Qwen3-1.7B-Base ← *Qwen3-4B-Instruct-2507*				Time per step (s)			
Batch size	Method	Tokens	k tok/s	Generation	Update	Other	Total
256	Vanilla	710	3.8	54	36	14	104
	STOPD	533	3.8	39	29	12	80
	Speedup	1.3×	1.0×	1.4×	1.2×	1.1×	**1.3×**
2048	Vanilla	820	5.5	308	232	92	632
	STOPD	595	5.1	240	171	66	478
	Speedup	1.4×	0.9×	1.3×	1.4×	1.4×	**1.3×**

F. Batch Size

Figure 12 shows the accuracy and cost of instruction following at both batch sizes of Section 5.5.

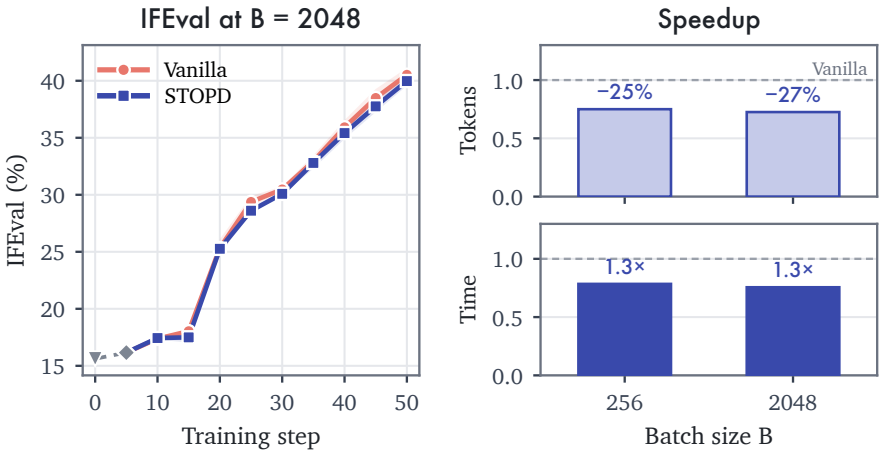


Figure 12 | Instruction following at $B = 256$ and $B = 2048$: IFEval against training step at $B = 2048$ (left), and tokens and training time relative to Vanilla (right).

G. Per-Benchmark Accuracy

Figures 13–16 break the averages of Figures 2 and 4 into their benchmarks, against training step: the base model sits at step 0 and the shared warmup checkpoint at step 25, and each point averages the

three seeds with the range across seeds shaded. Instruction following has a single benchmark, IFEval, shown in Figure 2.

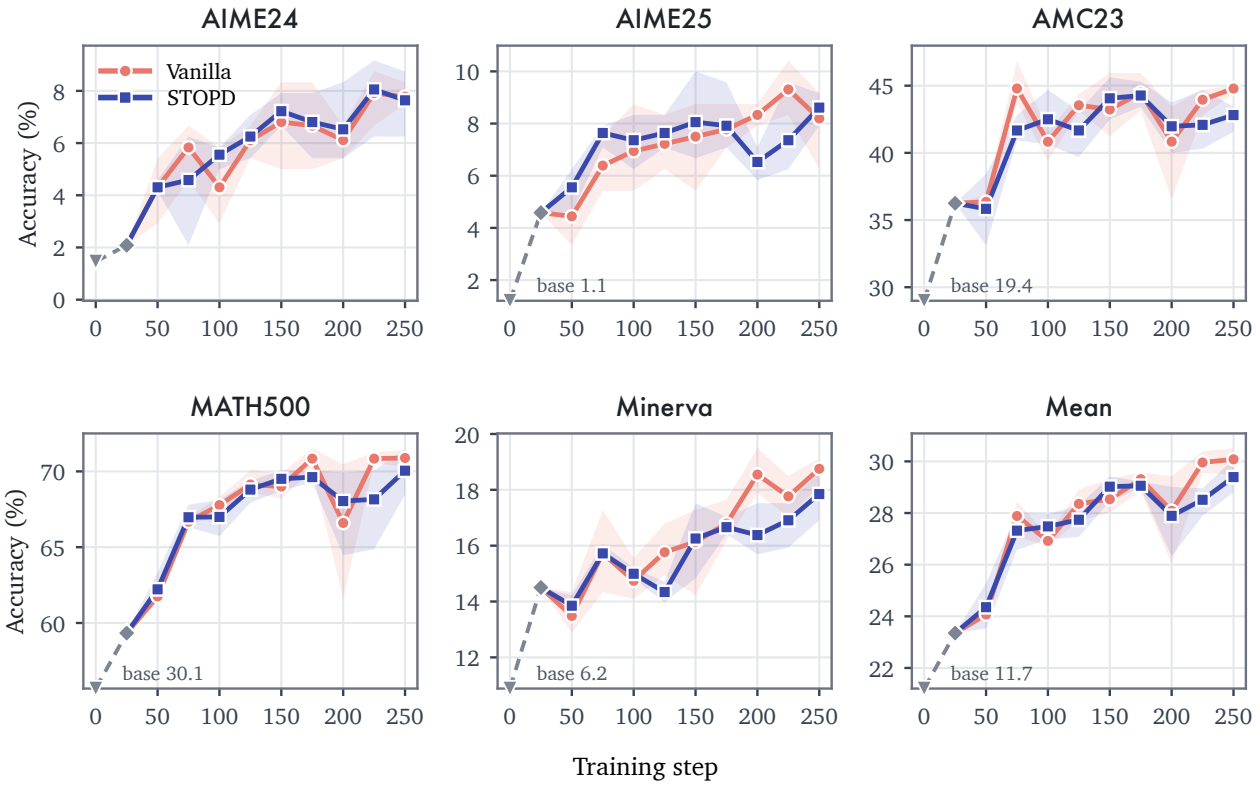


Figure 13 | Mathematics, Qwen3-1.7B-Base ← Qwen3-4B-Instruct-2507: accuracy per benchmark against training step.

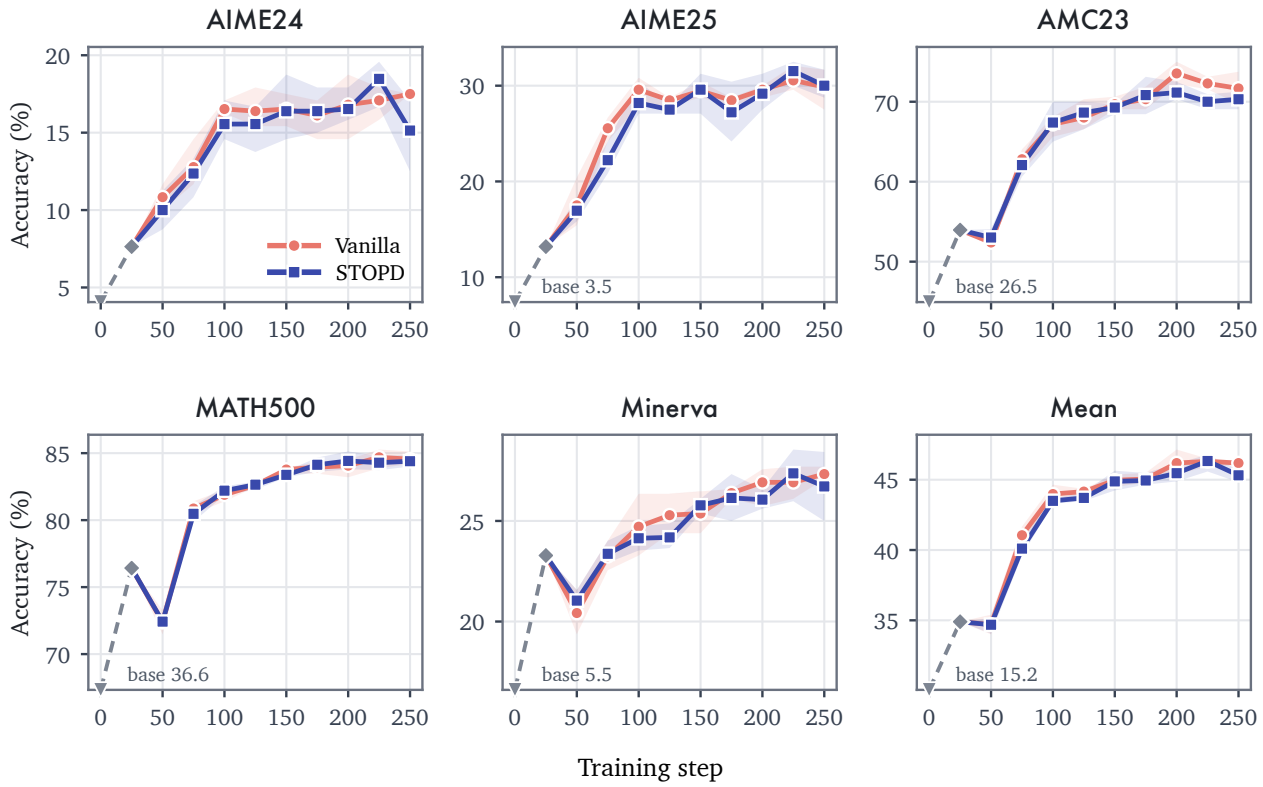


Figure 14 | Mathematics, Qwen3-4B-Base ← Qwen3-30B-A3B-Instruct-2507: accuracy per benchmark against training step.

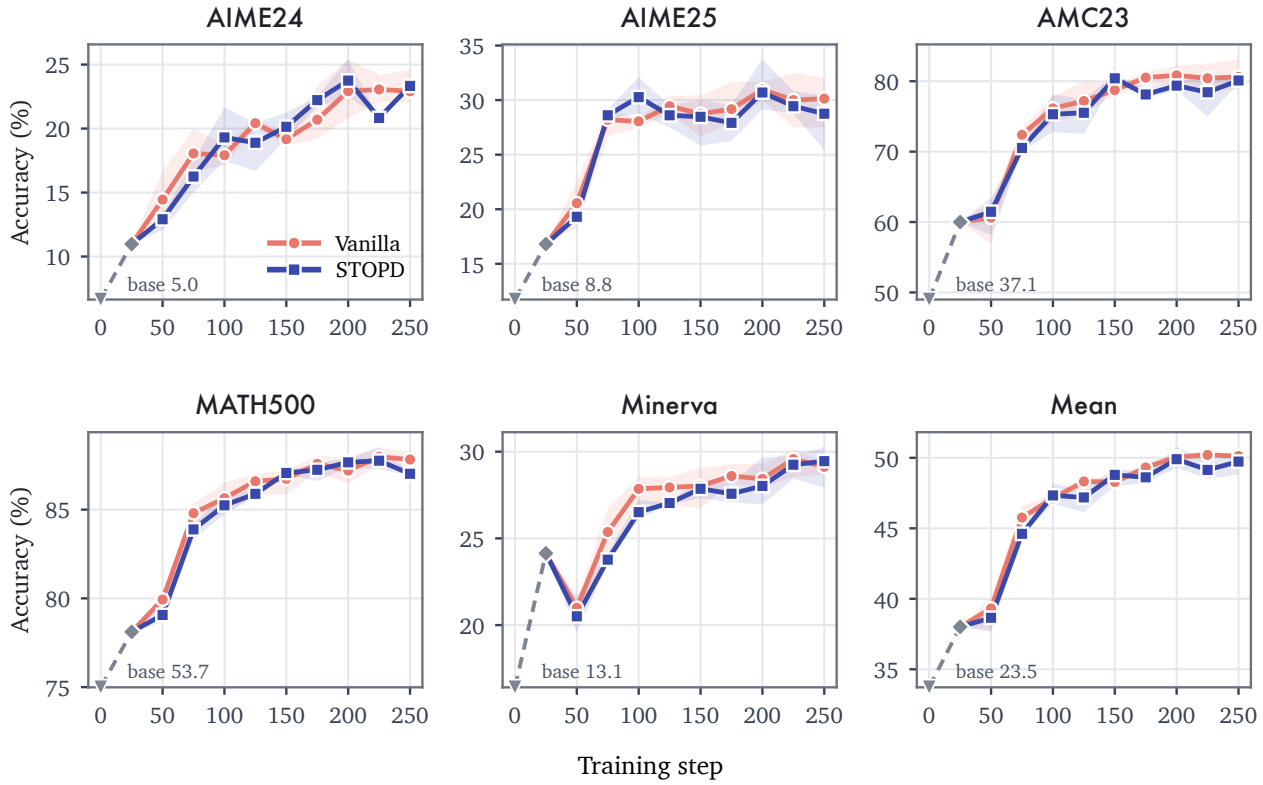


Figure 15 | Mathematics, Qwen3-8B-Base ← Qwen3-30B-A3B-Instruct-2507: accuracy per benchmark against training step.

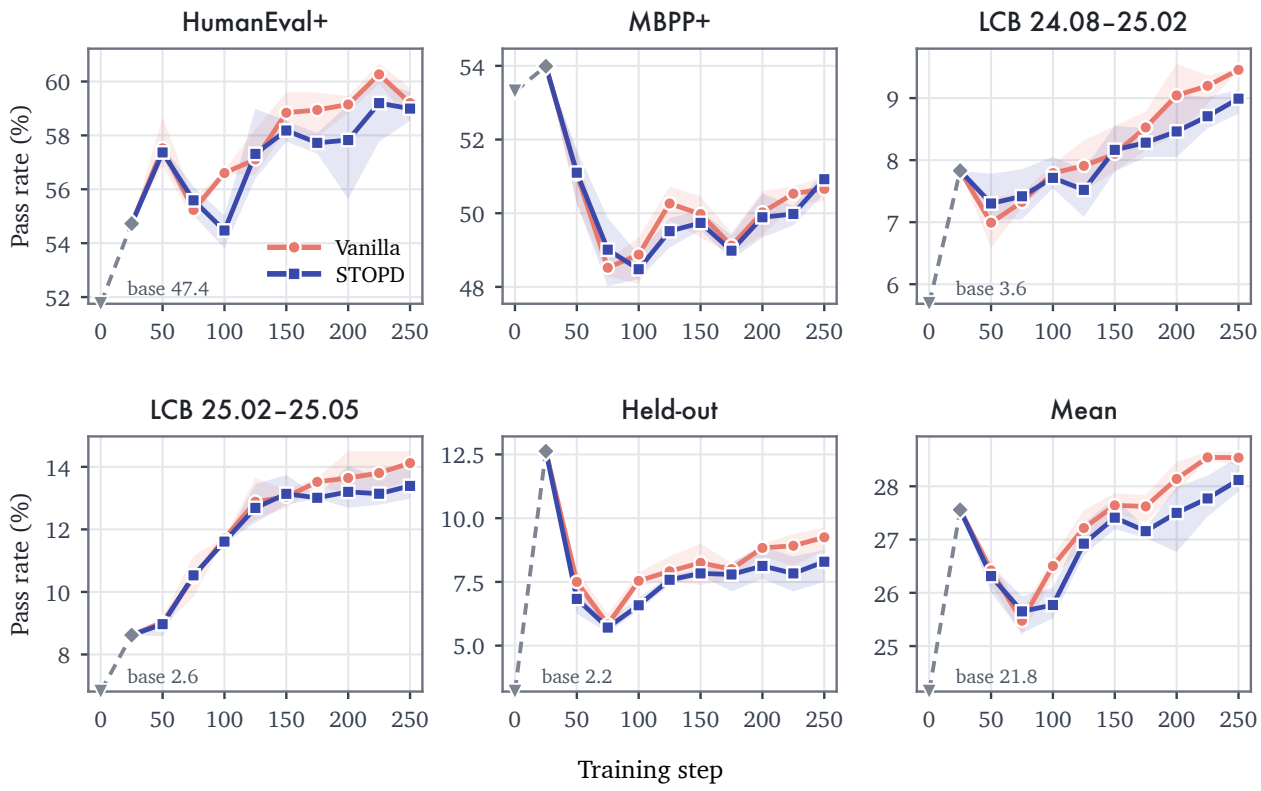


Figure 16 | Code, Qwen3-1.7B-Base ← Qwen3-4B-Instruct-2507: pass rate per benchmark.